

Diseño y simulación de un robot manipulador serial de seis grados de libertad para automatización logística.

Ignacio Martín Duci
ignacio8martin@gmail.com

Proyecto Final de Estudios (00441),
Facultad de Ingeniería,
Universidad Nacional de Cuyo,

Mendoza, Argentina

Mayo 2026, versión v003

Presentación parcial del Proyecto Final de Estudios para evaluación de avances

Agradecimientos

[completar]

Resumen

[completar]

Índice

1. Introducción	6
1.1. Motivación y mercado	6
1.2. Propuesta	6
2. Objetivos y beneficiarios	9
3. Descripción de la tarea y especificaciones del robot	11
4. Diseño conceptual preliminar	13
5. Modelado cinemático	15
5.1. Modelado cinemático	15
5.2. Análisis y verificación de espacio de trabajo	17
5.3. Cinemática inversa	17
5.4. Análisis de singularidades	24
6. Planificación de trayectorias	26
6.1. Generación de trayectorias	26
6.2. Escalamiento de velocidades y aceleraciones	26
7. Diseño y adaptación de CAD	28
7.1. Visualización del modelo en MATLAB	28
7.2. Diseño de piezas en <i>Fusion 360</i>	28
7.2.1. Base	29
7.2.2. Eslabón 1	29
7.2.3. Eslabón 2	30
7.2.4. Eslabón 3	30
7.2.5. Eslabón 4	30
7.2.6. Eslabón 5	31
7.2.7. Eslabón 6 – Herramienta	31
7.3. Diseño alternativo: apertura unidireccional de eslabones	32
8. Simulación y verificación de tarea	34
9. Integración en entorno ROS2	35
A. Verificación y Validación de Módulos de Software de MATLAB	36
A.1. Cinemática Inversa	36
A.2. Singularidades	37
A.3. Generación de trayectorias	37
A.4. Nombre del Módulo 1	38
A.5. Nombre del Módulo 1	38
B. Códigos MATLAB	39
B.1. <code>main.m</code>	40
B.2. <code>flags.m</code>	41
B.3. <code>config.m</code>	42
B.4. <code>robot_1.m</code>	42
B.5. <code>inv_kinematics.m</code>	43
B.6. <code>gen_traj.m</code>	58
B.7. <code>tarea.m</code>	62
B.8. <code>ik_comparison.m</code>	63
B.9. <code>scale.m</code>	68
B.10. <code>sym_1.m</code>	69
B.11. <code>test_workspace.m</code>	70
B.12. <code>test_inv_kinematics.m</code>	71
B.13. <code>test_singularities.m</code>	74

B.14.test_gen_traj.m	76
B.15.myAnimate.m	77
B.16.grafQ.m	86
B.17.grafQaE.m	87
B.18.grafScaling.m	89

1. Introducción

1.1. Motivación y mercado

La automatización e informatización de procesos de servicios es una trayectoria que, con el pasar de los años, aumenta su presencia en los diferentes mercados, brindando un aumento de la eficiencia, precisión, seguridad, operatividad y trazabilidad. Estos beneficios que brinda la implementación de tecnologías también quitan competitividad a aquellas entidades que, por disponibilidad, costo o falta de personal capacitado, no pueden implementar.

Al revisar datos aportados por la consultora *Grand View Research*, para analizar las proyecciones de mercado 2023-2030 para la robótica aplicada a depósitos logísticos o su traducción al inglés *warehouses* para el mundo [1], Estados Unidos [2], China [3], Latinoamérica [4] y Brasil [5] pueden elaborarse las siguientes conclusiones.

1. Todos los mercados proyectan un crecimiento anual compuesto de entre 17 % y 22.1 % lo cual se considera por encima de un escenario favorable.
2. Comparativamente, Latinoamérica proyecta un escenario de crecimiento mayor al de Estados Unidos, China y al mundo en totalidad.
3. Al analizar un país limítrofe que reporte datos como Brasil, puede verse que proyecta un crecimiento similar al de China en el período de tiempo analizado.

Bajo estas conclusiones, puede entenderse que el mercado continuará creciendo, y que países con menores niveles de automatización e informatización serán los que presentarán mejores oportunidades para este campo. Si se considera la situación actual de Argentina, orientada a un modelo productivo con menor carga impositiva, puede posicionarse como plataforma estratégica para fabricar y exportar robótica de servicio.

En el contexto de la automatización de los procesos logísticos, la implementación de robots manipuladores es esencial para lograr movimientos en el espacio con alto grado de flexibilidad que permitan la reorganización de paquetes con alta precisión y eficiencia.

El desarrollo desde cero de un robot tipo serie de seis grados de libertad presenta una excelente oportunidad para integrar los conocimientos adquiridos en la carrera. También se presentarán desafíos que requerirán expandir conocimientos y aplicar nuevos conceptos y técnicas con criterio. La robótica serie es un campo profundamente estudiado, por lo que podrá accederse a una gran cantidad de información ya existente.

1.2. Propuesta

En el presente trabajo se desarrolla el diseño y la simulación de un robot manipulador serial de seis grados de libertad orientado a una aplicación específica: la transferencia de objetos entre contenedores (*bin-to-bin transfer*) de hasta 3 kg.

Esta aplicación se plantea como un caso representativo dentro de procesos de manipulación en depósitos logísticos automatizados (*micro-fulfillment warehouses*), permitiendo validar el desempeño del sistema en tareas típicas de pick-and-place.

La selección del caso de estudio se inspira en desarrollos industriales existentes, como el presentado en [YouTube](#). No obstante, la solución propuesta en este trabajo es de desarrollo propio, enfocada en el modelado cinemático, la generación de trayectorias y la simulación del comportamiento del manipulador.

En la figura 1 puede verse un esquema de la aplicación descrita.

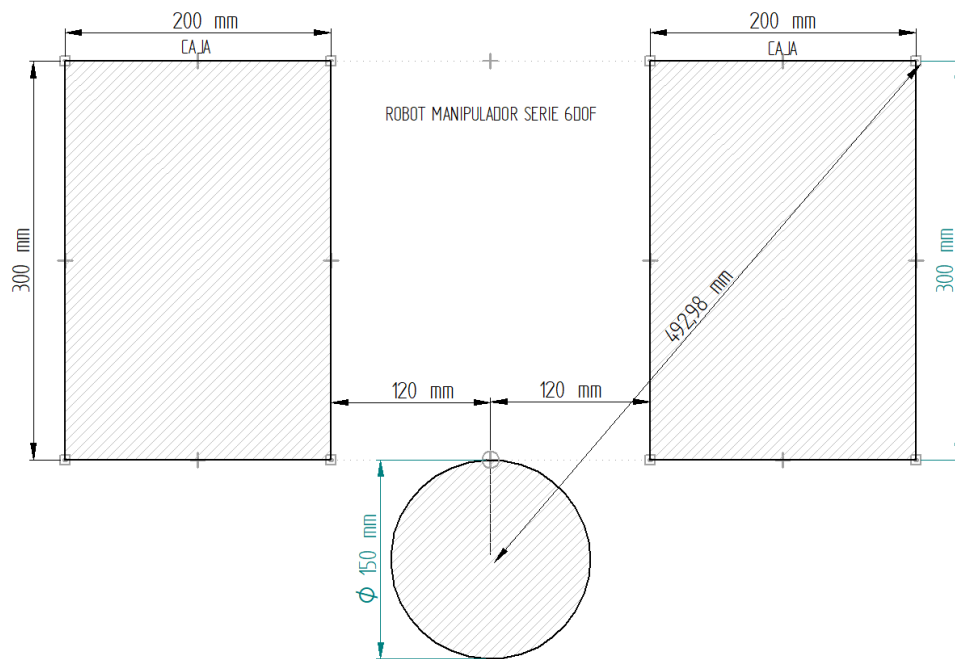


Figura 1: Esquema de la aplicación - vista superior.

Mínimamente, se propone que el robot tenga un alcance de 500 mm con una capacidad de carga útil de hasta 3 kg, similar a las características del robot comercial [UR3e](#). En la figura 2 se puede ver un esquema general de la morfología del robot.

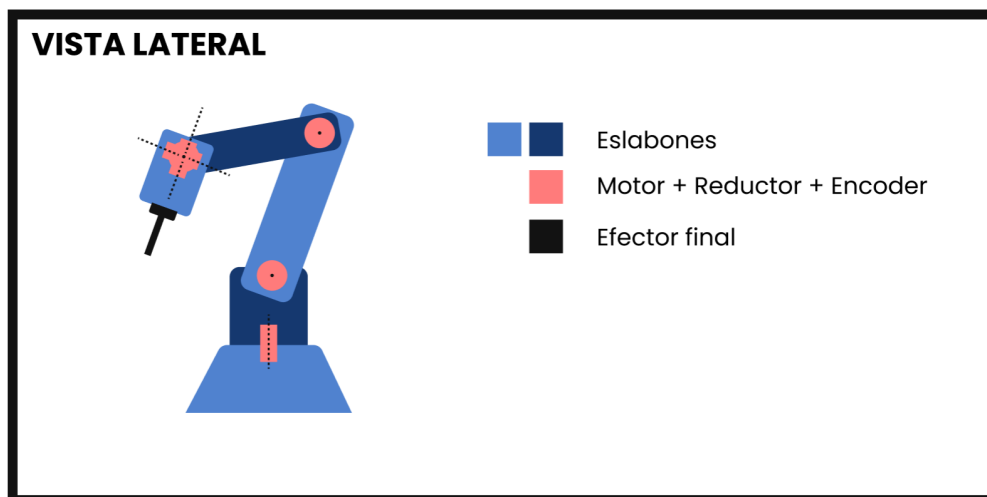


Figura 2: Esquema del robot tipo serie de 6 GDL propuesto.

La tarea del robot consiste cronológicamente en lo siguiente:

1. Roto-traslación a postura de espera de forma rápida.
2. Roto-traslación a postura sobre caja origen a alta velocidad.
3. Roto-traslación para *picking* a baja velocidad.
4. Roto-traslación hacia postura sobre caja origen a baja velocidad.
5. Roto-traslación hacia postura sobre caja destino a alta velocidad.
6. Roto-traslación para *placing* a baja velocidad.

7. Roto-traslación hacia postura sobre caja destino a baja velocidad.

8. Repetir.

Dentro de la bibliografía a consultar inicialmente se mencionan los libros [7], [8], [9], [10], [11], [12] y [13].

2. Objetivos y beneficiarios

Objetivo mínimo: Diseñar y simular un robot manipulador de seis ejes tipo serie con 500 mm de alcance y 3 kg de carga útil destinado a tareas de *bin-to-bin transfer* capaz de mover 300 elementos por hora.

Los siguientes objetivos se desarrollan en dos etapas: los ítems 1 a 6 corresponden a la implementación y validación en entorno MATLAB, mientras que los ítems subsiguientes se desarrollan en el entorno ROS 2.

1. Definir las especificaciones del robot y su tarea con un primer acercamiento hacia las normas de seguridad de máquinas, robótica y colaboración humano-máquina.
2. Realizar un diseño conceptual preliminar.
3. Analizar y validar el espacio de trabajo.
4. Resolver analíticamente y validar la cinemática inversa.
5. Implementar y validar un programa de generación de trayectorias que incorpore distintas funcionalidades, permitiendo la selección de modos de interpolación (cartesiana y articular) y la gestión de singularidades.
6. Implementar, simular y validar la ejecución de la tarea definida, asegurando su correcto desempeño y cumplimiento de los objetivos planteados.
7. Desarrollar un modelo CAD para simulaciones cinemáticas, asegurando la correspondencia entre los sistemas de referencia de cada componente y los marcos definidos mediante la convención Denavit-Hartenberg, a fin de garantizar consistencia geométrica y fidelidad en la representación del sistema en entornos de simulación.
8. Desarrollar el modelo del robot en formato URDF a partir de la parametrización cinemática definida (DH), asegurando la correcta correspondencia entre eslabones y articulaciones.
9. Integrar y verificar la coherencia de las geometrías CAD como representaciones visuales del modelo.
10. Desarrollar y validar un nodo en ROS 2 encargado de publicar estados articulares del manipulador, permitiendo definir configuraciones de prueba y animar el modelo en el entorno de simulación.
11. Desarrollar y validar un nodo o servicio en ROS 2 que implemente la cinemática inversa previamente desarrollada del manipulador, permitiendo calcular configuraciones articulares a partir de poses objetivo del efector final.
12. Desarrollar un nodo de ROS 2 encargado de la generación de trayectorias articulares y/o cartesianas a partir del programa desarrollado previamente.
13. Implementar y validar la ejecución completa de la tarea, integrando los nodos de cinemática inversa, generación de trayectorias y publicación de estados articulares.
14. Realizar simulaciones físicas en el entorno de ROS 2 para evaluar el comportamiento del robot bajo nuevos fenómenos.
15. Desarrollar una interfaz gráfica en ROS 2 que permita la visualización en tiempo real de variables cinemáticas y dinámicas del robot, así como el ingreso de comandos de alto nivel para la ejecución de tareas dentro del entorno ROS 2.

Posibles beneficiarios:

1. **Bodegas y agro-industria mendocina** que requieren automatizar el empaque y la manipulación de insumos sin invertir en robots industriales de alto costo.

2. **Centros de *micro-fulfillment* y PYMES logísticas** donde el brazo puede aumentar el *pick-rate* y reducir tiempos de entrega.
3. **Institutos y universidades locales** (UNCuyo, ITU, UTN) que obtendrán una plataforma didáctica *open-source* para prácticas de mecatrónica y control.

3. Descripción de la tarea y especificaciones del robot

La tarea del manipulador consiste en la transferencia de objetos entre dos contenedores de dimensiones y posiciones predefinidas. Esto puede verse en las figuras 3 y 4 donde también quedan definidos puntos clave de la trayectoria que deberá seguir el robot conforme a lo definido en la sección 1.2.

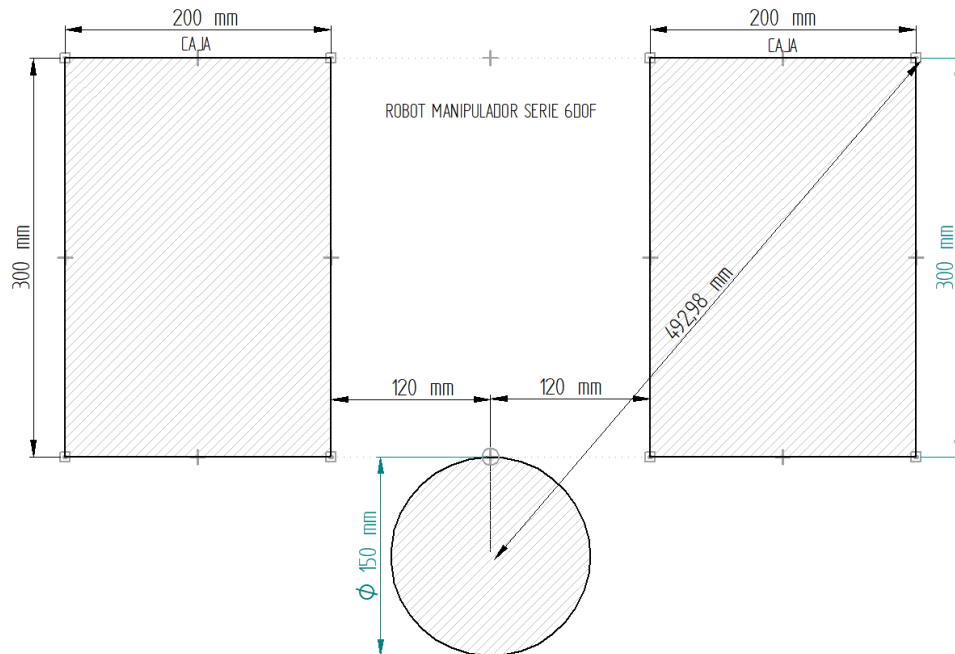


Figura 3: Vista superior del robot y la tarea

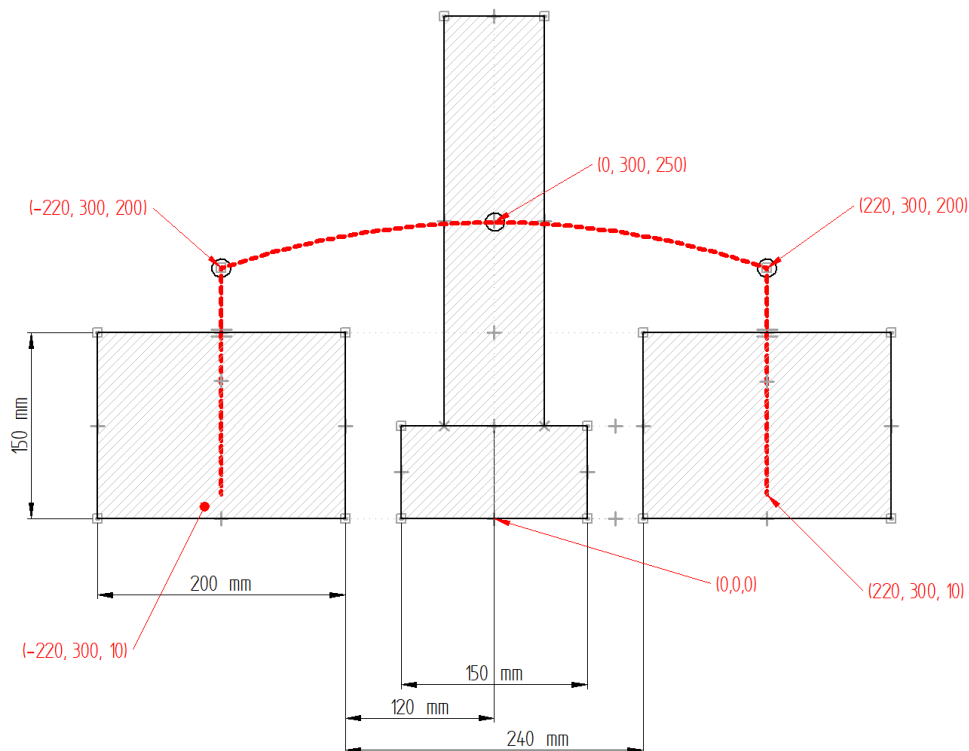


Figura 4: Vista lateral del robot y la tarea

Los objetos serán tomados mediante un sistema de sujeción por vacío (ventosa - *vacuum gripper*) y depositados mediante la liberación del mismo, esto permite relajar los requerimientos de precisión y repetibilidad del manipulador.

Quedan definidas las especificaciones principales del manipulador en función de la tarea descrita¹:

- Alcance: 500.00 mm
- Capacidad de carga útil: 3.00 kg
- Precisión: 0.5 mm
- Repetibilidad: 0.5 mm
- Grados de libertad (GDL - *DOF*): 6.
- Rango articulaciones $1:5 \pm 2\pi$
- Rango articulación final *libre*

Se definen también otras especificaciones y condiciones del entorno, posiblemente útiles para desarrollos posteriores:

- Temperatura ambiente entre 5 y 35 °C.
- Humedad relativa $\leq 70\%$.
- Bajo nivel de polvo ambiental.
- Iluminación 100-150 lx.
- Entorno sin vibraciones.
- Protección IP54

¹Se considera un alcance y una capacidad de carga superiores a los requeridos por la tarea, con el fin de incorporar un margen de diseño que garantice un funcionamiento adecuado del sistema y permita su adaptación a tareas similares.

4. Diseño conceptual preliminar

Con el objetivo de tener un punto de partida en cuanto a la morfología y dimensiones del robot, tomando como referencia al robot [UR3e](#), se define la siguiente estructura, morfología y dimensiones:

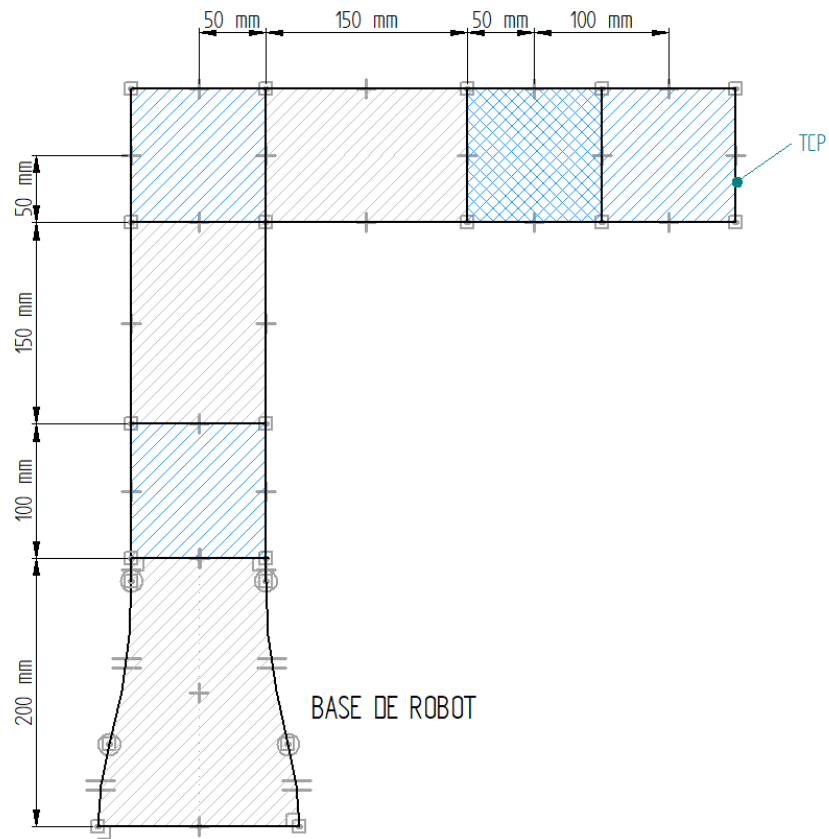


Figura 5: Vista lateral 1 de un esquema del robot

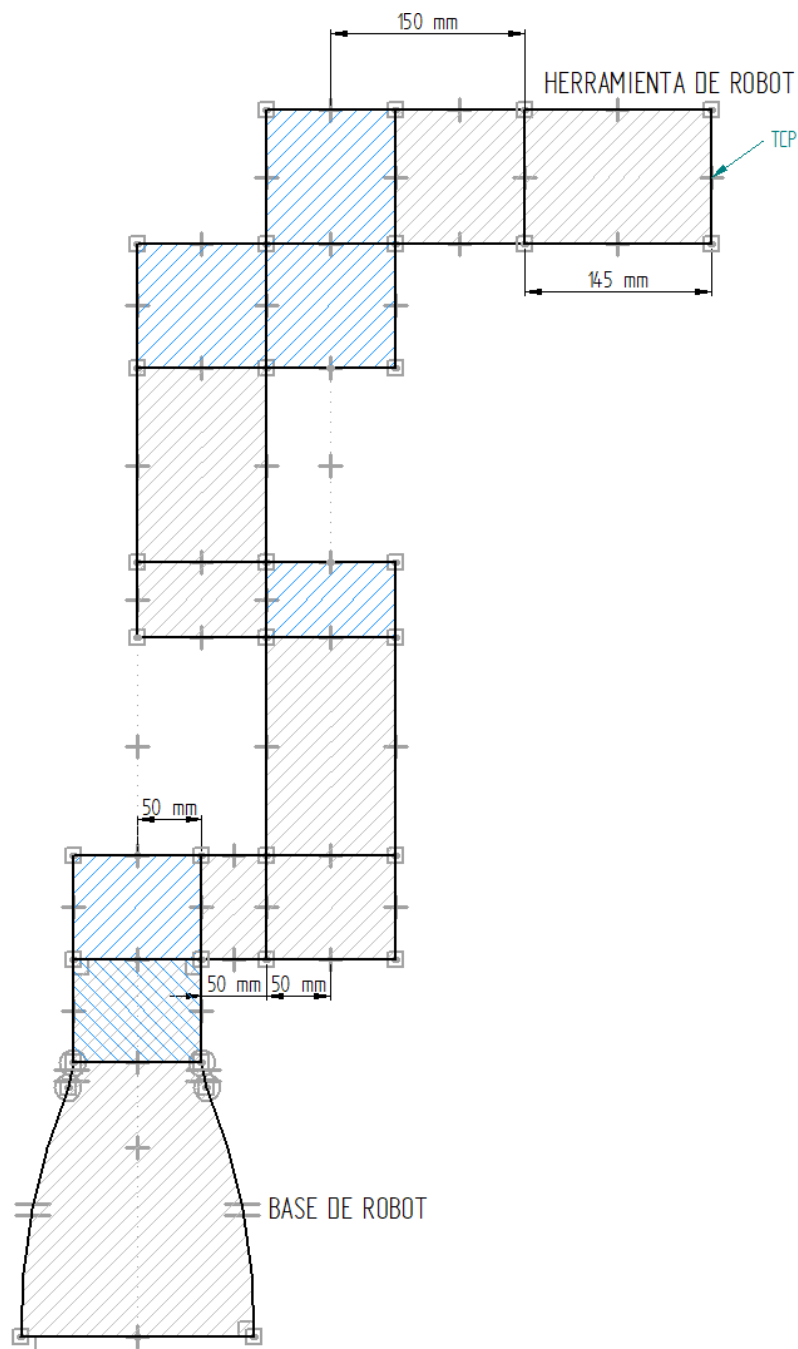


Figura 6: Vista lateral 2 del un esquema del robot

A partir de estas imágenes se obtiene la información necesaria para la elaboración del modelo cinemático. Se ha incorporado una base para permitir que el robot opere de forma segura por encima de las cajas y una herramienta final tipo ventosa (*vacuum-gripper*), para esta última se toma como referencia el modelo *EPick with one suction cup 145x83 mm* del fabricante [Robotiq](#).

5. Modelado cinemático

En esta sección se aborda el modelado cinemático del manipulador, así como la resolución de la cinemática inversa y la planificación de trayectorias necesarias para su operación.

El modelado cinemático permite establecer la relación entre las variables articulares del robot y la posición y orientación de su efector final. A partir de este modelo, se plantea la resolución de la cinemática inversa, con el objetivo de determinar las configuraciones articulares requeridas para alcanzar posiciones específicas dentro del espacio de trabajo.

Finalmente, se desarrolla la planificación de trayectorias, mediante la cual se define la evolución temporal del movimiento del manipulador entre distintas configuraciones, garantizando continuidad, suavidad y seguridad en su ejecución.

5.1. Modelado cinemático

Para obtener el modelo cinemático se aplica el algoritmo Denavit-Hartenberg estándar el cual consiste en lo siguiente:

1. Enumerar los $n+1$ eslabones desde 0 hasta n , comenzando desde la base (eslabón fijo) y terminando en el efector final.
2. Identificar los ejes de cada articulación. Si es rotacional, será el eje de giro; si es prismática, será el eje a lo largo del cual se produce el desplazamiento.
3. Enumerar los ejes de 1 a n , comenzando desde el que une el eslabón base con el eslabón 1.
4. Para i de 0 a $n-1$, situar el eje Z_1 en el eje de la articulación $i+1$.
5. El eje Z_n se colocará en el extremo del último eslabón en la misma dirección que el eje Z_{n-1} .
6. Situar el origen del sistema de la base $\{S_0\}$ en cualquier punto del eje Z_0 .
7. Para $i = 1$ a n , ubicar el sistema $\{S_i\}$ en la intersección entre el eje Z_i y la recta perpendicular común a los ejes Z_i y Z_{i-1} . En caso de que los ejes Z_i y Z_{i-1} se intersecten, el sistema $\{S_i\}$ se posiciona en dicho punto de intersección.
8. Para $i = 1$ a n , definir el eje X_i a partir del punto donde se estableció el sistema $\{S_i\}$, sobre la recta perpendicular común a los ejes Z_i y Z_{i-1} . Si los ejes Z_i y Z_{i-1} se intersectan, el eje X_i debe ser perpendicular a ambos. Su orientación puede elegirse de forma arbitraria.
9. El eje X_0 puede definirse libremente, siendo conveniente alinearlos con el eje X_1 .
10. Para $i = 0$ a n , definir el eje Y_i de manera que, junto con los ejes X_i y Z_i , forme un sistema de referencia dextrógiro.

Luego se determinan los parámetros que permitirán elaborar una matriz que define al robot en cuestión de la siguiente forma:

Para $i = 1$ a n :

1. θ_i : Ángulo alrededor del eje Z_{i-1} , desde el eje X_{i-1} hasta el eje X_i .
2. d_i : Distancia a lo largo del eje Z_{i-1} , desde el origen del sistema $i - 1$ hasta el eje X_i .
3. a_i : Distancia a lo largo del eje X_i , desde el eje Z_{i-1} hasta el eje Z_i .
4. α_i : Ángulo alrededor del eje X_i , desde el eje Z_{i-1} hasta el eje Z_i .

Donde:

- a_i : Longitud del eslabón.
- α_i : Ángulo de torsión del eslabón.
- d_i : Longitud articular.

- θ_i : Ángulo articular.

Utilizando esta información, los parámetros de Denavit-Hartenberg pueden organizarse en la siguiente forma general:

i	θ_i	d_i	a_i	α_i
1	θ_1	d_1	a_1	α_1
2	θ_2	d_2	a_2	α_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	θ_n	d_n	a_n	α_n

(1)

Finalmente, se ha conseguido describir la geometría del robot y la interrelación entre sus eslabones mediante un algoritmo. El beneficio de esta representación radica en que permite determinar la posición y orientación de cada sistema de referencia a partir del anterior y, por consiguiente, obtener la posición y orientación del efector final en función de las variables articulares. A continuación, se presenta de forma general la matriz de transformación homogénea que relaciona cada sistema de referencia con el anterior.

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Puede entonces determinarse, para un robot de 6 grados de libertad, la posición y orientación del efector final en función de las variables articulares mediante la transformación sucesiva del sistema base hasta el sistema herramienta.

$${}^{base}T_{tcp} = {}^bT_0 {}^0T_1 \dots {}^5T_6 {}^6T_{tcp} \quad (3)$$

De esta forma quedará definida **el modelo cinemático directo**.

Se obtiene información sobre las dimensiones del robot de la figura 5 y 6 y se elabora el siguiente esquema simplificado con información solo esencial al aplicar el algoritmo DH:

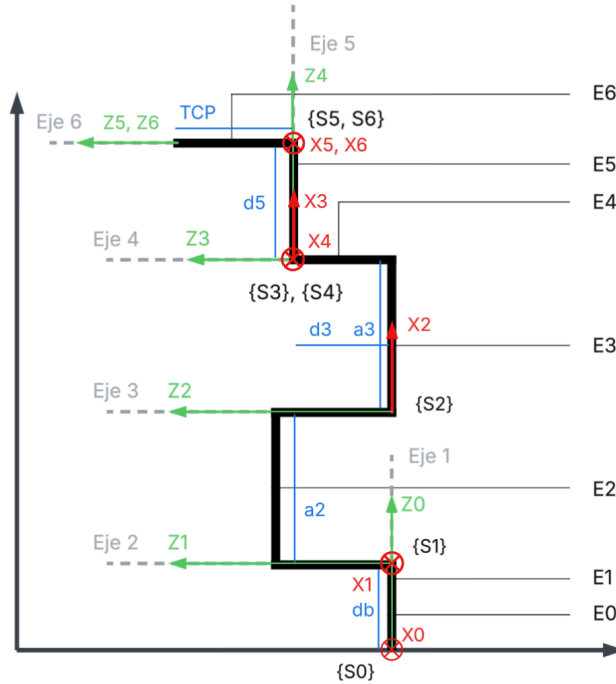


Figura 7: Algoritmo DH aplicado a un esquema simplificado del robot

Teniendo en cuenta que las articulaciones son de tipo rotacional, la variable articular será θ para todos los casos. Se define la matriz Denavit-Hartenberg a partir de la información de la figura 7 y del procedimiento definido en la explicación del algoritmo:

i	$\theta_i[rad]$	$d_i[m]$	$a_i[m]$	$\alpha_i[rad]$
1	q_1	0,25	0	$-\frac{\pi}{2}$
2	$q_2 - \frac{\pi}{2}$	0	0,25	0
3	q_3	0,295	0,25	0
4	$q_4 + \frac{\pi}{2}$	0	0	$\frac{\pi}{2}$
5	q_5	0,1	0	$-\frac{\pi}{2}$
6	q_6	0	0	0

(4)

Notas:

- Mediante la letra "q" se simboliza el parámetro variable.
- La suma o resta a valores de q , como lo es en los casos 2 y 4, dan un offset de posición que modifica el punto cero del robot.

5.2. Análisis y verificación de espacio de trabajo

Para verificar el alcance máximo y la tarea de nuestro robot, simplemente buscamos una posición de alto estiramiento ($q = [variable, \frac{\pi}{2}, 0, -\frac{\pi}{2}, -\frac{\pi}{2}, 0]$) y se ingresan valores progresivos completando el giro de la primera articulación. Luego, esto produce el siguiente resultado (script `test_workspace.m`):

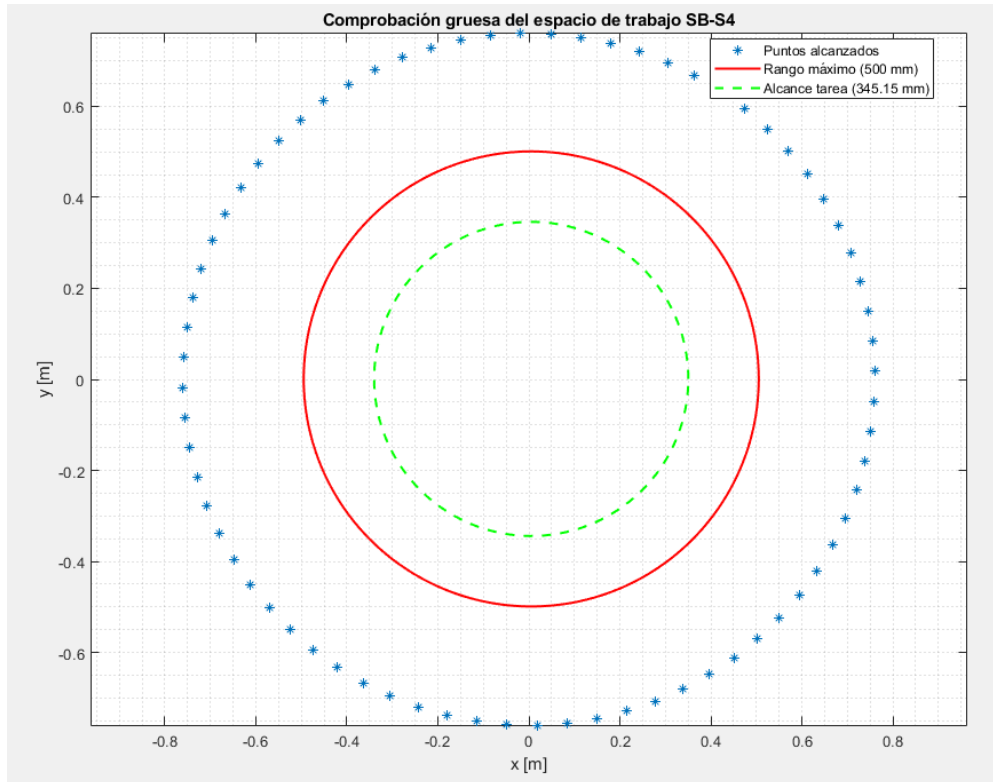


Figura 8: Verificación de espacio de trabajo en plano XY

5.3. Cinemática inversa

En el desarrollo cinemático de todo robot es fundamental elaborar el modelo de cinemática inversa, el cual permite determinar las variables articulares necesarias para alcanzar una posición y orientación

deseadas del efector final. En este trabajo, la resolución se aborda mediante un enfoque analítico, obteniendo expresiones en forma cerrada que relacionan directamente las variables del espacio cartesiano con las variables articulares, evitando el uso de métodos numéricos iterativos.

Se parte de la observación de que el manipulador no cuenta con una muñeca esférica; es decir, los tres ejes correspondientes a la muñeca no se intersectan en un mismo punto, lo que no nos permite hacer uso del método de Pieper.

En esta resolución, se implementan métodos algebraicos para separar incógnitas y determinarlas en función de parámetros conocidos y variables ya determinadas.

Dado que se ha definido el modelo cinemático directo al haber elaborado la matriz Denavit-Hartenberg se conocen todas las matrices de transformación homogénea, particularmente:

$$T_{tcp}^{base}, T_0^{base}, T_{tcp}^6 \quad (5)$$

Al desacoplar la base y la herramienta final se puede conocer:

$$T_6^0 = [T_0^{base}]^{-1} T_{tcp}^{base} [T_{tcp}^6]^{-1} \quad (6)$$

También se sabe que:

$$T_6^0 = T_1^0 T_2^1 T_3^2 T_4^3 T_5^4 T_6^5 \quad (7)$$

En esta ecuación hay una alta complejidad en el lado derecho (RHS) y una baja complejidad en el lado izquierdo (LSH) por lo que se equilibra de la siguiente manera:

$$[T_1^0]^{-1} T_6^0 [T_6^5]^{-1} = T_2^1 T_3^2 T_4^3 T_5^4 \quad (8)$$

De forma genérica la matriz de transformación homogénea que vincula al sistema $i - 1$ con el sistema i es:

$$T_i^{i-1} = \begin{bmatrix} c_i & -s_i c_j & s_i s_j & a_i c_i \\ s_i & c_i c_j & -s_i s_j & a_i s_i \\ 0 & s_j & c_j & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9)$$

Donde:

$$\begin{aligned} s_i &:= \sin(\theta_i) \\ c_i &:= \cos(\theta_i) \\ s_j &:= \sin(\alpha_i) \\ c_j &:= \cos(\alpha_i) \end{aligned} \quad (10)$$

NOTA: El subíndice i y j no indican sistemas diferentes sino que i hace referencia al ángulo θ y j al ángulo α para simplificar la nomenclatura de las funciones.

Cada matriz de transformación homogénea esta compuesta por cuatro partes:

$$[T_i^{i-1}] = \begin{bmatrix} R_{3x3} & p_{3x1} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se puede obtener la matriz de transformación homogénea inversa mediante la siguiente expresión:

$$\begin{aligned} [T_i^{i-1}]^{-1} &= \begin{bmatrix} R^T & -R^T T \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ R^T &= \begin{bmatrix} c_i & s_i & 0 \\ -s_i c_j & c_i c_j & s_j \\ s_i s_j & -c_i s_j & c_j \end{bmatrix} \\ p &= \begin{bmatrix} a_i c_i \\ a_i s_i \\ d_i \end{bmatrix} \end{aligned}$$

$$-R^T p = \begin{bmatrix} -c_i & -s_i & 0 \\ +s_i c_j & -c_i c_j & -s_j \\ -s_i s_j & +c_i s_j & -c_j \end{bmatrix} \begin{bmatrix} a_i c_i \\ a_i s_i \\ d_i \end{bmatrix} = \begin{bmatrix} -a_i \\ -d_i s_j \\ -d_i c_j \end{bmatrix}$$

Y se obtiene la matriz de transformación homogénea inversa:

$$[T_i^{i-1}]^{-1} = \begin{bmatrix} c_i & s_i & 0 & -a_i \\ -s_i c_j & c_i c_j & s_j & -d_i s_j \\ s_i s_j & -c_i s_j & c_j & -d_i c_j \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se toma esta plantilla de matriz y la plantilla de la matriz 9, se reemplazan valores y se simplifican para formar las matrices de transformación homogéneas presentes en la ecuación 8.

$$[T_1^0]^{-1} = \begin{bmatrix} c_1 & s_1 & 0 & 0 \\ 0 & 0 & -1 & d_1 \\ -s_1 & c_1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_6^5]^{-1} = \begin{bmatrix} c_6 & s_6 & 0 & 0 \\ -s_6 & c_6 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_2^1] = \begin{bmatrix} s_2 & c_2 & 0 & a_2 s_2 \\ -c_2 & s_2 & 0 & -a_2 c_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_3^2] = \begin{bmatrix} c_3 & -s_3 & 0 & a_3 c_3 \\ s_3 & c_3 & 0 & a_3 s_3 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_4^3] = \begin{bmatrix} -s_4 & 0 & c_4 & 0 \\ c_4 & 0 & s_4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_5^4] = \begin{bmatrix} c_5 & 0 & -s_5 & 0 \\ s_5 & 0 & c_5 & 0 \\ 0 & -1 & 0 & d_5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$[T_6^0] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & x \\ r_{21} & r_{22} & r_{23} & y \\ r_{31} & r_{32} & r_{33} & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

La ecuación 8, con sus matrices ya determinadas, se carga a un script de MATLAB `sym_1.m` para, mediante trabajo simbólico, determinar la expresión final simplificada de cada lado de la ecuación; se obtiene:

$$\text{LHS} = \begin{bmatrix} c_6(r_{11}c_1 + r_{21}s_1) - s_6(r_{12}c_1 + r_{22}s_1) & c_6(r_{12}c_1 + r_{22}s_1) + s_6(r_{11}c_1 + r_{21}s_1) & r_{13}c_1 + r_{23}s_1 & xc_1 + ys_1 \\ r_{32}s_6 - r_{31}c_6 & -r_{32}c_6 - r_{31}s_6 & -r_{33} & d_1 - z \\ c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) & c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) & r_{23}c_1 - r_{13}s_1 & yc_1 - xs_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{RHS} = \begin{bmatrix} c_5 c_{234} & -s_{234} & -s_5 c_{234} & a_3 s_{23} + a_2 s_2 + d_5 s_{234} \\ c_5 s_{234} & c_{234} & -s_5 s_{234} & -a_3 c_{23} - a_2 c_2 - d_5 c_{234} \\ s_5 & 0 & c_5 & d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Es importante resaltar que los valores r_{ij} , x , y y z , son valores conocidos; son la información de entrada al cálculo de la cinemática inversa. También es necesario aclarar que $c_{abc} = \cos(a + b + c)$ y $s_{abc} = \sin(a + b + c)$.

Las ecuaciones formadas por los elementos de las matrices LHS y RHS son los siguientes:

$$c_6(r_{11}c_1 + r_{21}s_1) - s_6(r_{12}c_1 + r_{22}s_1) = c_5 c_{234} \quad (11)$$

$$c_6(r_{12}c_1 + r_{22}s_1) + s_6(r_{11}c_1 + r_{21}s_1) = -s_{234} \quad (12)$$

$$r_{13}c_1 + r_{23}s_1 = -s_5 c_{234} \quad (13)$$

$$xc_1 + ys_1 = a_3 s_{23} + a_2 s_2 + d_5 s_{234} \quad (14)$$

$$r_{32}s_6 - r_{31}c_6 = c_5 s_{234} \quad (15)$$

$$-r_{32}c_6 - r_{31}s_6 = c_{234} \quad (16)$$

$$-r_{33} = -s_5 s_{234} \quad (17)$$

$$d_1 - z = -a_3 c_{23} - a_2 c_2 - d_5 c_{234} \quad (18)$$

$$c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) = s_5 \quad (19)$$

$$c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) = 0 \quad (20)$$

$$r_{23}c_1 - r_{13}s_1 = c_5 \quad (21)$$

$$yc_1 - xs_1 = d_3 \quad (22)$$

Se toma la ecuación formada por la posición 3:4 (fila 3, columna 4, ecuación 22):

$$yc_1 - xs_1 = d_3 \quad (23)$$

$$yc_1 - xs_1 - d_3 = 0 \quad (24)$$

$$A = y, B = -x, C = -d_3 \quad (25)$$

$$Ac_1 + Bs_1 + C = 0 \quad (26)$$

Se aplica la técnica de sustitución por la tangente de medio ángulo:

$$\begin{aligned} t &= \tan \frac{\theta_1}{2} \\ \cos(\theta_1) &= \frac{1 - t^2}{1 + t^2} \\ \sin(\theta_1) &= \frac{2t}{1 + t^2} \end{aligned} \quad (27)$$

$$A \frac{1-t^2}{1+t^2} + B \frac{2t}{1+t^2} + C = 0 \quad (28)$$

$$A(1-t^2) + B2t + C(1+t^2) = 0 \quad (29)$$

$$A - At^2 + B2t + C + Ct^2 = 0 \quad (30)$$

$$t^2(C - A) + t(2B) + (C + A) = 0 \quad (31)$$

$$t_{1,2} = \frac{-2B \pm \sqrt{4B^2 - 4(C - A)(C + A)}}{2(C - A)} \quad (32)$$

De esta última ecuación se pueden determinar dos valores que conducen a dos valores de θ_1 mediante:

$$\theta_{1-1,2} = 2 \arctan(t_{1,2}) \quad (33)$$

La sustitución de medio ángulo $t = \tan(\theta/2)$ establece una relación biyectiva para $\theta \in (-\pi, \pi)$. De este modo, al recuperar el ángulo mediante $\theta = 2 \tan^{-1}(t)$, se extiende el rango de la función arco tangente y se evita la ambigüedad de cuadrante.

Una vez determinados ambos valores para q_1 , se analiza la ecuación formada por los miembros de la posición 3:2 (fila 3, columna 2, ecuación 20):

$$c_6(r_{22}c_1 - r_{12}s_1) + s_6(r_{21}c_1 - r_{11}s_1) = 0 \quad (34)$$

$$A = -(r_{22}c_1 - r_{12}s_1) \quad (35)$$

$$B = r_{21}c_1 - r_{11}s_1$$

$$Ac_6 + Bs_6 = 0 \quad (36)$$

$$\frac{s_6}{c_6} = t_6 \quad (37)$$

$$\theta_6 = \text{atan2}(t_6) \quad (38)$$

De esta última ecuación se obtiene un θ_6 por cada valor de θ_1 .

Se toma la ecuación 3:1 (fila 3, columna 1, ecuación 19) y la ecuación 3:3 (fila 3, columna 3, ecuación 21):

$$s_5 = c_6(r_{21}c_1 - r_{11}s_1) - s_6(r_{22}c_1 - r_{12}s_1) \quad (39)$$

$$c_5 = r_{23}c_1 - r_{13}s_1$$

$$\frac{s_5}{c_5} = t_5 \quad (40)$$

$$\theta_5 = \text{atan2}(t_5) \quad (41)$$

Para determinar correctamente los valores de θ_5 es fundamental mantener trazabilidad entre los resultados obtenidos para las variables articulares que intervienen. De esta manera se obtiene un valor de θ_5 por cada combinación de θ_1 y θ_6 .

Desde las ecuaciones formadas en las posiciones 1:3 (fila 1, columna 3, ecuación 13) y 2:3 (fila 2, columna 3, ecuación 17):

$$\begin{aligned} -s_5c_{234} &= r_{13}c_1 + r_{23}s_1 \\ -s_5s_{234} &= -r_{33} \end{aligned} \quad (42)$$

$$\begin{aligned} A = c_{234} &= \frac{r_{13}c_1 + r_{23}s_1}{-s_5} \\ B = s_{234} &= \frac{r_{33}}{s_5} \end{aligned} \quad (43)$$

Se continúa tomando las ecuaciones de las posiciones 1:4 (fila 1, columna 4, ecuación 14) y 2:4 (fila 2, columna 4, ecuación 18):

$$\begin{aligned} xc_1 + ys_1 &= a_3s_{23} + a_2s_2 + d_5s_{234} \\ d_1 - z &= -a_3c_{23} - a_2c_2 - d_5c_{234} \end{aligned} \quad (44)$$

Se reemplaza lo obtenido previamente en estas ecuaciones:

$$\begin{aligned} xc_1 + ys_1 &= a_3s_{23} + a_2s_2 + d_5B \\ d_1 - z &= -a_3c_{23} - a_2c_2 - d_5A \end{aligned} \quad (45)$$

Para simplificar, se define:

$$\begin{aligned} a &= -d_5A + z - d_1 \\ b &= d_5B - xc_1 - ys_1 \end{aligned} \quad (46)$$

Y queda de la siguiente forma:

$$\begin{aligned} a_3c_{23} &= -a_2c_2 + a \\ a_3s_{23} &= -a_2s_2 - b \end{aligned} \quad (47)$$

Al observar el lado izquierdo de cada ecuación, puede notarse que son las componentes de un vector, por lo tanto, podemos simplificar al elevar al sumar el cuadrado de cada lado entre sí.

$$(a_3c_{23})^2 + (a_3s_{23})^2 = (-a_2c_2 + a)^2 + (-a_2s_2 - b)^2 \quad (48)$$

$$a_3^2 = a_2^2c_2^2 - 2a_2c_2a + a^2 + a_2^2s_2^2 + 2a_2s_2b + b^2 \quad (49)$$

$$a_3^2 = a_2^2 + a^2 + b^2 + 2a_2s_2b - 2a_2c_2a \quad (50)$$

Se define:

$$\begin{aligned} C &= a_2^2 + a^2 + b^2 - a_3^2 \\ D &= 2a_2b \end{aligned} \quad (51)$$

$$E = -2a_2a$$

$$Ds_2 + Ec_2 + C = 0 \quad (52)$$

Se aplica la sustitución por la tangente de medio ángulo, de forma análoga a lo realizado en el primer ángulo y se obtiene:

$$t^2(C - E) + t(2D) + (C + E) = 0 \quad (53)$$

$$t_{1,2} = \frac{-2D \pm \sqrt{4D^2 - 4(C - E)(C + E)}}{2(C - E)} \quad (54)$$

$$\theta_{2-1,2} = 2\arctan(t_{1,2}) \quad (55)$$

Se reutilizan las ecuaciones de 47 para formar:

$$t_{23} = \frac{s_{23}}{c_{23}} = \frac{-a_2s_2 - b}{-a_2c_2 + a} \quad (56)$$

$$\theta_{23} = \operatorname{atan2}(t_{23}) \quad (57)$$

$$\theta_3 = \theta_{23} - \theta_2 \quad (58)$$

Se recuerda que debe mantenerse la trazabilidad de resultados para evitar cruzar soluciones. Finalmente, se reutilizan las ecuaciones de 43:

$$c_{234} = \frac{r_{13}c_1 + r_{23}s_1}{-s_5} \quad (59)$$

$$s_{234} = \frac{-r_{33}}{s_5}$$

$$t_{234} = \frac{r_{33}}{-(r_{13}c_1 + r_{23}s_1)} \quad (60)$$

$$\theta_{234} = \text{atan2}(t_{234}) \quad (61)$$

$$\theta_4 = \theta_{234} - \theta_2 - \theta_3 \quad (62)$$

Se deben tener en cuenta las mismas consideraciones sobre la trazabilidad de soluciones que se ha mencionado previamente.

Se han obtenido todas las soluciones geométricas posibles (8) desde las cuales puede elaborarse un cuadro de referencia para seguir la traza de cada solución más claramente:

	q_1	q_2	q_3	q_4	q_5	q_6
Solución 1	q_1	q_2	q_3	q_4	q_5	q_6
Solución 2	q_1	q_2^I	q_3^I	q_4^I	q_5	q_6
Solución 3	q_1	q_2^{II}	q_3^{II}	q_4^{II}	q_5^I	q_6^I
Solución 4	q_1	q_2^{III}	q_3^{III}	q_4^{III}	q_5^I	q_6^I
Solución 5	q_1^I	q_2^{IV}	q_3^{IV}	q_4^{IV}	q_5^{II}	q_6^{II}
Solución 6	q_1^I	q_2^V	q_3^V	q_4^V	q_5^{II}	q_6^{II}
Solución 7	q_1^I	q_2^{VI}	q_3^{VI}	q_4^{VI}	q_5^{III}	q_6^{III}
Solución 8	q_1^I	q_2^{VII}	q_3^{VII}	q_4^{VII}	q_5^{III}	q_6^{III}

El robot a desarrollar cuenta con articulaciones rotativas con rango $[-2\pi, 2\pi]$, lo que permite realizar un giro completo en cada sentido. Esto permite una mayor flexibilidad en la planificación de trayectorias, ya que existen múltiples configuraciones articulares equivalentes para alcanzar una misma pose del efector final. Esto provoca una *periodicidad en las soluciones*, es decir, se puede alcanzar el mismo punto de un círculo al girar 15° o -345° .

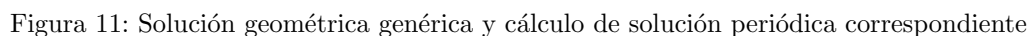
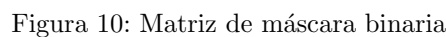
En primer lugar deben conocerse la cantidad de soluciones totales considerando periodicidad en articulaciones, dado que el rango total es, $\pm k\pi$ donde $k = 2$, se duplicarán las soluciones por cada articulación que tenga periodicidad. En este caso, se consideran 6 articulaciones con periodicidad igual a 2 (NOTA: se considera que el rango de la última articulación es $\pm 2\pi$, en realidad es una articulación libre por lo que conduciría a infinitas soluciones), así:

$$\text{Cantidad total de soluciones} = \text{Cantidad de soluciones geométricas} \times 2^6 = 8 \times 2^6 = 512 \quad (63)$$

La metodología elegida para formar soluciones coherentes es:

1. Se define una matriz vacía cuyo tamaño corresponde al número total de soluciones, dado por: *Cantidad de soluciones geométricas* $\times 2^6$.
2. Se construye una matriz máscara binaria de tamaño 64×6 , cuyas filas codifican la selección entre el valor original de cada articulación (0) y su equivalente periódico (1), garantizando así la generación de todas las combinaciones posibles. Esto se itera para cada solución geométrica encontrada.

- A continuación, se presentan imágenes de la forma genérica de cada matriz.



Las singularidades representan configuraciones en las que un manipulador robótico pierde uno o más grados de libertad en el espacio cartesiano, independientemente del movimiento que se imponga en el espacio articular. En un robot de seis grados de libertad, estas configuraciones se identifican a partir del análisis del Jacobiano geométrico $\mathbf{J}(\mathbf{q})$, cuya pérdida de rango completo indica la presencia de una singularidad. Formalmente, una configuración $\mathbf{q} \in \mathbb{R}^6$ es singular si y solo si

$$\det(\mathbf{J}(\mathbf{q})) = 0. \quad (64)$$

En estas configuraciones, el robot exhibe dos consecuencias prácticas críticas: la imposibilidad de generar velocidades cartesianas en determinadas direcciones y la amplificación teóricamente ilimitada de velocidades articulares ante pequeñas velocidades cartesianas en las direcciones próximas al espacio nulo del Jacobiano. Por este motivo, la detección y el tratamiento de singularidades son requisitos indispensables en la planificación de trayectorias y en el control de manipuladores industriales.

Al realizar un análisis geométrico, se identifican tres tipos de singularidades: de hombro, de codo y de muñeca. Matemáticamente, cada una corresponde a una condición específica sobre las variables articulares del robot.

- **Singularidad de muñeca:** ocurre cuando el eje de la articulación 4 y el eje de la articulación 6 se alinean, colapsando un grado de libertad en la orientación del efector final. La condición que la define es

$$q_5 = k\pi, \quad k \in \mathbb{Z}. \quad (65)$$

- **Singularidad de codo:** ocurre cuando los ejes de las articulaciones 2 y 4 se alinean, de modo que ambas articulaciones producen el mismo efecto rotacional y el Jacobiano pierde rango. La condición que la define es

$$q_3 = k\pi, \quad k \in \mathbb{Z}. \quad (66)$$

- **Singularidad de hombro:** ocurre cuando el punto de intersección de los ejes articulares 5 y 6 se encuentra sobre el plano que forman los ejes articulares 1 y 2. La forma analítica de esta explicación es hacer que la proyección vertical total de a_2 , a_3 y d_5 sea nula:

$$a_2 \sin(q_2) + a_3 \sin(q_2 + q_3) + d_5 \sin(q_2 + q_3 + q_4) = 0 \quad (67)$$

La verificación de las singularidades se explica en [A.2](#)

6. Planificación de trayectorias

6.1. Generación de trayectorias

La generación de trayectorias constituye una etapa fundamental en el control de manipuladores robóticos, ya que permite definir la evolución temporal de las variables articulares o del efector final entre configuraciones iniciales y finales previamente establecidas. Este proceso no solo implica determinar el camino geométrico a seguir, sino también garantizar que el movimiento cumpla con restricciones físicas del sistema, tales como límites de velocidad, aceleración y suavidad.

El cálculo de la trayectoria se realiza dentro del script `gen_traj.m` el cual recibe los siguientes argumentos:

- **R**: definición del robot.
- **cpoints_input**: matriz de puntos vía en coordenadas cartesianas, uno por fila, con una séptima columna que indica el modo de interpolación para llegar a ese punto (0: espacio articular, 1: espacio cartesiano).
- **q0**: coordenada articular inicial del robot.
- **qddmax, qddmax**: velocidades y aceleraciones máximas articulares.
- **verbose**: controla la verbosidad de la cinemática inversa interna.
- **vmax, amax, wmax, alphamax**: velocidades y aceleraciones lineales y angulares máximas del efector final.

El trabajo comienza por analizar la séptima columna de cada fila (punto vía) del argumento **cpoints_input**, para determinar en qué modo de interpolación llegar a ese punto: 0 para interpolación en espacio articular y 1 para interpolación en espacio cartesiano.

Si el modo es 0, el primer paso es empaquetar todos los puntos vías consecutivos dentro de la matriz de *waypoints* para obtener una trayectoria suave y continua sin detención en cada punto vía dado originalmente. En segundo lugar, estos puntos se transforman al espacio articular mediante la aplicación del cálculo analítico de la cinemática inversa provisto en el script `inv_kinematics.m`. Los puntos vía en el espacio articular son enviados al interpolador de espacio articular provisto por el toolbox **mstraj**.

En el otro caso, cuando el modo es 1, la interpolación se da en el espacio cartesiano por lo que simplemente se envía el punto objetivo al interpolador en espacio cartesiano provisto en el toolbox *ctray* en conjunto con la ley de movimiento definida en *s*. La ley de movimiento se calcula de modo que el tiempo total del movimiento quede definido por el parámetro más comprometido (*vmax*, *amax*, *wmax*, *alphamax*).

En ambos casos, al calcular la trayectoria, esta se acopla a continuación de la trayectoria ya definida en una matriz. El último elemento de esta matriz se utiliza punto inicial de la siguiente interpolación, en el caso de que esté vacía, es decir, que sea la primera interpolación, se utilizará el argumento **q0**.

La validación del funcionamiento del módulo de generación de trayectorias se presenta en el Apéndice [A.3](#).

6.2. Escalamiento de velocidades y aceleraciones

Una vez generada una trayectoria preliminar, es necesario verificar que el perfil de velocidades y aceleraciones resultante respeta los límites cinemáticos del robot, tanto en el espacio articular como en el espacio cartesiano. Para esto se realiza el siguiente procedimiento de dos etapas.

Primera etapa: generación preliminar y cómputo del factor de escala

Se genera la trayectoria con los parámetros nominales. A continuación, la función auxiliar `compute_scale` calcula numéricamente las velocidades y aceleraciones articulares mediante diferencias finitas y, a través del Jacobiano del robot $\mathbf{J}(\mathbf{q})$, obtiene las velocidades y aceleraciones cartesianas:

$$\mathbf{v} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}, \quad \mathbf{a} = \mathbf{J}(\mathbf{q}) \ddot{\mathbf{q}}. \quad (68)$$

Con estos valores se determina si algún límite es superado y en qué medida. La clave del método reside en la relación entre el factor de escala temporal k y sus efectos sobre la cinemática: si se ejecuta el mismo recorrido en k veces más tiempo, las velocidades se reducen en un factor $1/k$ y las aceleraciones en $1/k^2$. Por esto, los factores de escala individuales se calculan de forma diferente según se trate de una restricción de velocidad o de aceleración:

$$k_{\dot{q}} = \max_i \frac{|\dot{q}_i|_{\max}}{\dot{q}_{i,\max}}, \quad k_{\ddot{q}} = \sqrt{\max_i \frac{|\ddot{q}_i|_{\max}}{\ddot{q}_{i,\max}}}, \quad (69)$$

y de manera análoga para las magnitudes cartesianas:

$$k_v = \frac{\|\mathbf{v}\|_{\max}}{v_{\max}}, \quad k_\omega = \frac{\|\boldsymbol{\omega}\|_{\max}}{\omega_{\max}}, \quad k_a = \sqrt{\frac{\|\mathbf{a}\|_{\max}}{a_{\max}}}, \quad k_\alpha = \sqrt{\frac{\|\boldsymbol{\alpha}\|_{\max}}{\alpha_{\max}}}. \quad (70)$$

La raíz cuadrada en los factores de aceleración refleja que, al estirar el tiempo por k , la aceleración se reduce en k^2 , por lo que para cancelar una violación de factor r en aceleración basta con $k = \sqrt{r}$. El factor de escala final del segmento es:

$$k = \max(1, k_{\dot{q}}, k_{\ddot{q}}, k_v, k_\omega, k_a, k_\alpha). \quad (71)$$

Si $k = 1$ no existe ninguna violación y la trayectoria preliminar es válida.

Segunda etapa: regeneración de la trayectoria

Si $k > 1$, se regenera la trayectoria con el tiempo extendido. En el caso de interpolación articular (**mstraj**), esto equivale a reducir los límites de velocidad articular a $\dot{\mathbf{q}}_{\max}/k$ y extender el tiempo de rampa a $t_{\text{acc}} \cdot k$, con lo que la trayectoria resultante cumple todos los límites. En el caso de interpolación cartesiana, simplemente se extiende el tiempo total a t_f a $k \cdot t_f$ y se regenera la ley de movimiento quintic. En este modo existe además una estimación inicial de t_f basada en las propiedades analíticas del perfil quintic empleado, que sirve como punto de partida para la verificación posterior.

La validación del funcionamiento del escalamiento de velocidades se presenta en el Apéndice [A.3](#).

7. Diseño y adaptación de CAD

El diseño tridimensional del robot constituye una etapa fundamental para la visualización y validación del modelo cinemático desarrollado. Con este objetivo, se elaboraron los modelos CAD de cada eslabón en el entorno *Fusion 360*, exportándolos posteriormente en formato STL para su integración en el entorno de simulación. Dado que las dimensiones y unidades de los archivos exportados no coinciden directamente con los parámetros del modelo DH, se desarrolló el script `scale.m`, que se encarga de escalar cada STL de forma que las dimensiones coincidan con las definidas modelo DH. Adicionalmente, durante el proceso de diseño se buscó maximizar el aprovechamiento del rango articular de cada articulación, procurando que la configuración mecánica permita explotar al máximo las capacidades de movimiento del robot sin incurrir en interferencias geométricas entre sus componentes.

7.1. Visualización del modelo en MATLAB

La figura 12 muestra el ensamblaje completo del robot renderizado en *MATLAB* mediante la toolbox de *Peter Corke*, con los marcos de referencia DH superpuestos sobre cada eslabón y con transparencia aplicada sobre las mallas STL, lo que permite verificar visualmente la alineación entre los ejes articulares y la geometría de cada pieza. Se incluyen también los sistemas del modelo Dh.

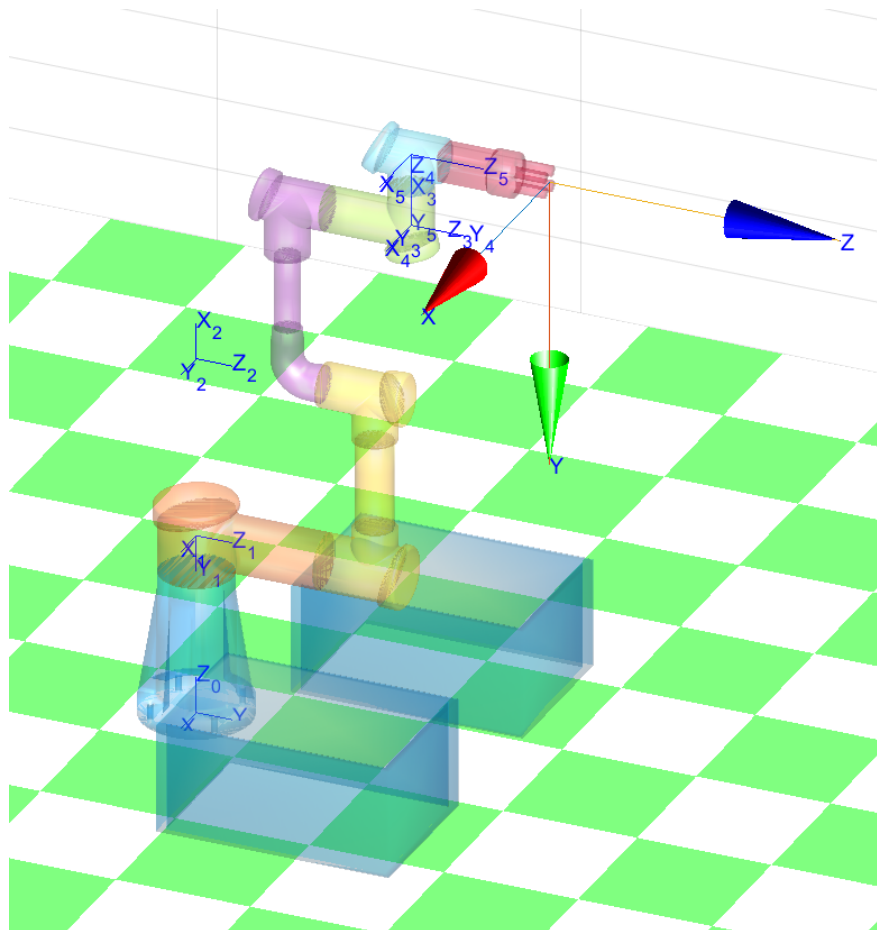


Figura 12: Modelo del robot renderizado en *MATLAB* con marcos de referencia DH.

7.2. Diseño de piezas en *Fusion 360*

A continuación se presentan los modelos de cada pieza diseñada en *Fusion 360*. Para cada componente se detallan las decisiones de diseño adoptadas en relación al aprovechamiento del rango articular y la evitación de interferencias geométricas. En todos los casos, el sistema de referencia de cada cuerpo se adapta de forma que resulte correlativo con el modelo DH, garantizando la coincidencia entre el eje de giro definido en la parametrización cinemática y el cuerpo sobre el cual dicho giro se produce.

7.2.1. Base

Para el diseño de la base se tienen en cuenta dos premisas. En primer lugar, se incrementa el grosor en la zona de apoyo con el objetivo de aumentar la estabilidad del conjunto. En segundo lugar, se incorpora una altura suficiente para posicionar al robot por encima de las cajas, tal como lo requiere la tarea descrita en la sección 3.

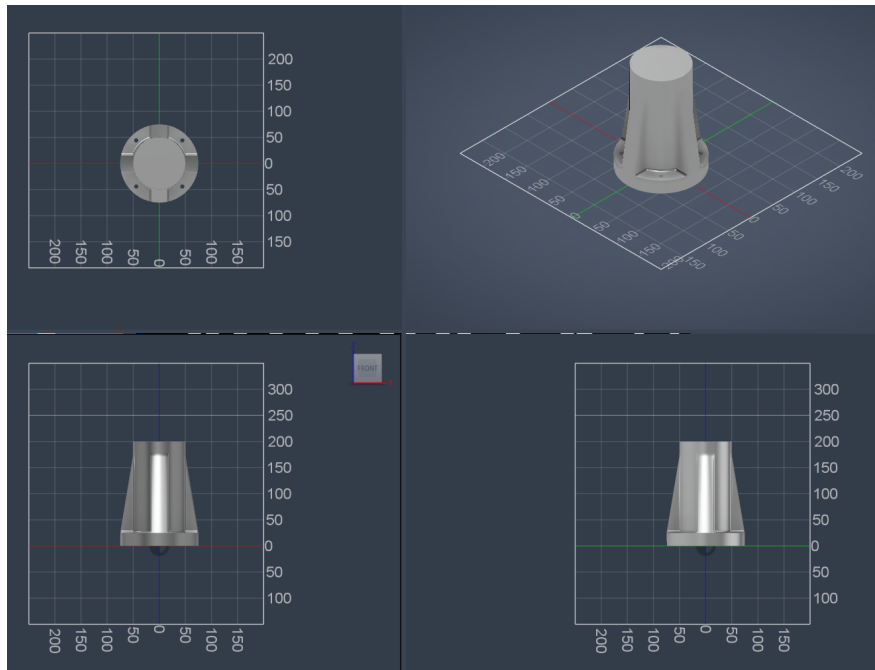


Figura 13: Diseño de la base en *Fusion 360*.

7.2.2. Eslabón 1

El eslabón 1 tiene como objetivo principal proveer una separación del eje central definido por la base, de forma que los eslabones posteriores cuenten con espacio suficiente para aprovechar su rango articular sin incurrir en interferencias geométricas.

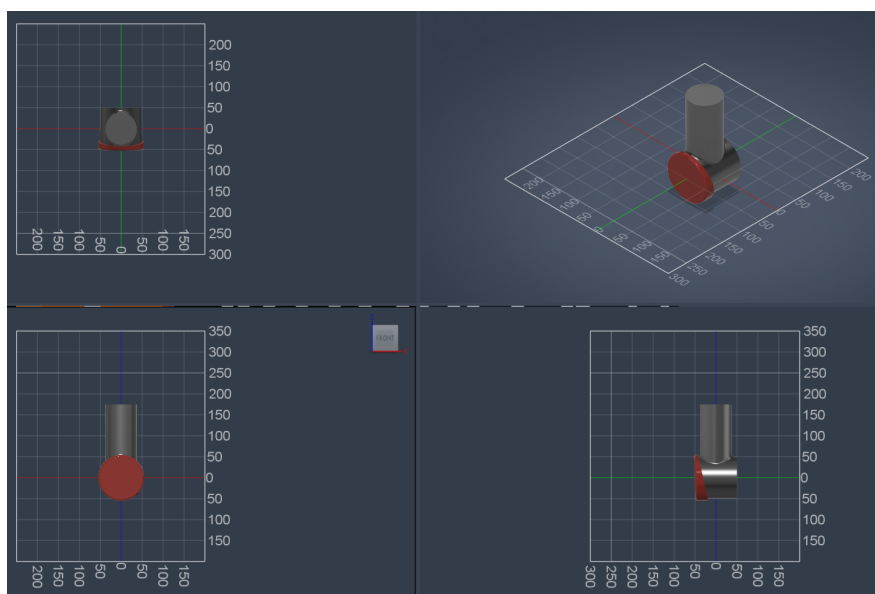


Figura 14: Diseño del eslabón 1 en *Fusion 360*.

7.2.3. Eslabón 2

El eslabón 2 cumple la misma función que el eslabón 1. La separación total requerida se distribuye entre ambos eslabones, logrando un equilibrio en la geometría del robot.

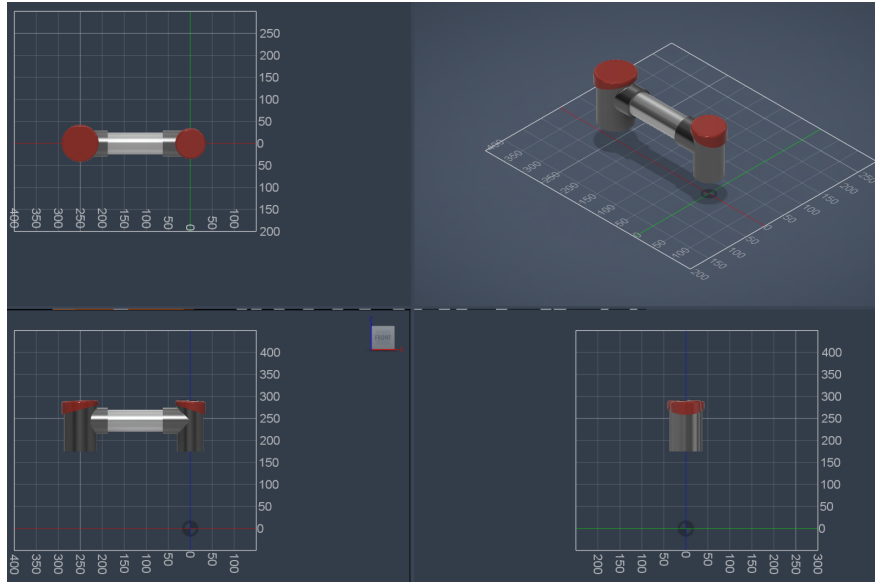


Figura 15: Diseño del eslabón 2 en *Fusion 360*.

7.2.4. Eslabón 3

El eslabón 3 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

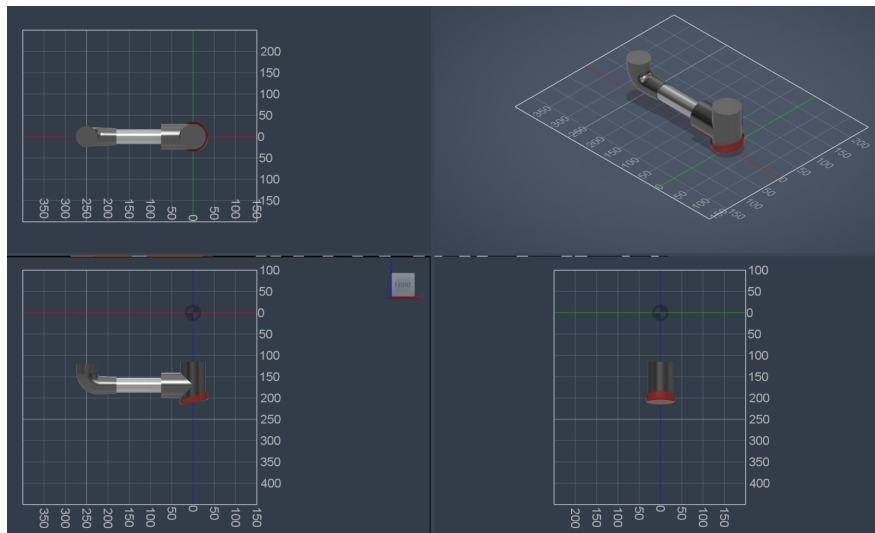


Figura 16: Diseño del eslabón 3 en *Fusion 360*.

7.2.5. Eslabón 4

El eslabón 4 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

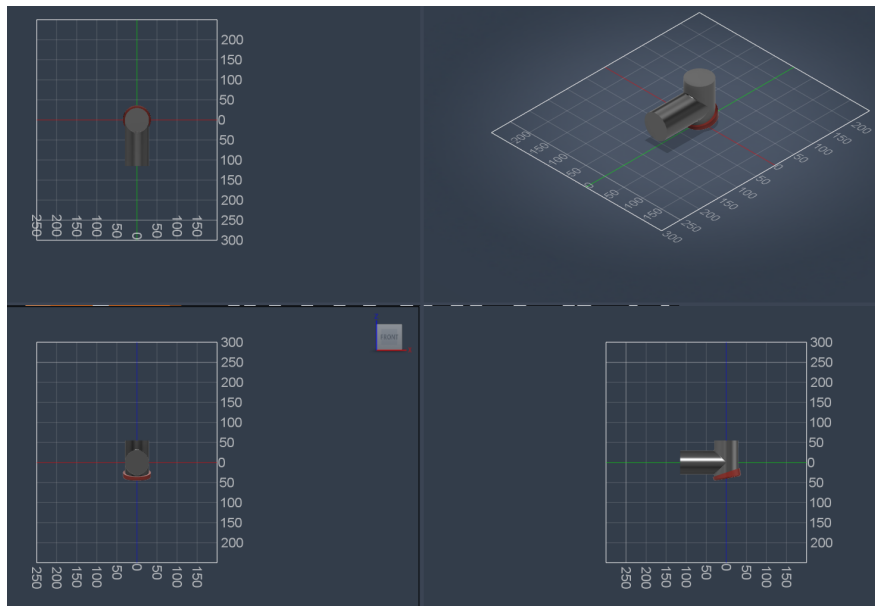


Figura 17: Diseño del eslabón 4 en *Fusion 360*.

7.2.6. Eslabón 5

El eslabón 5 no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

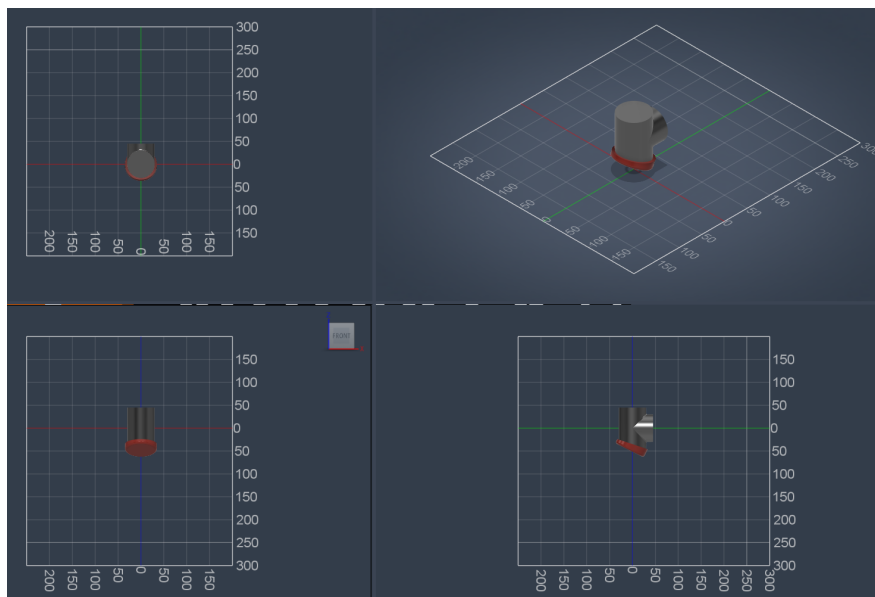


Figura 18: Diseño del eslabón 5 en *Fusion 360*.

7.2.7. Eslabón 6 – Herramienta

El eslabón 6 corresponde a la herramienta del sistema y no presenta decisiones de diseño particulares más allá de las consideraciones generales de la sección.

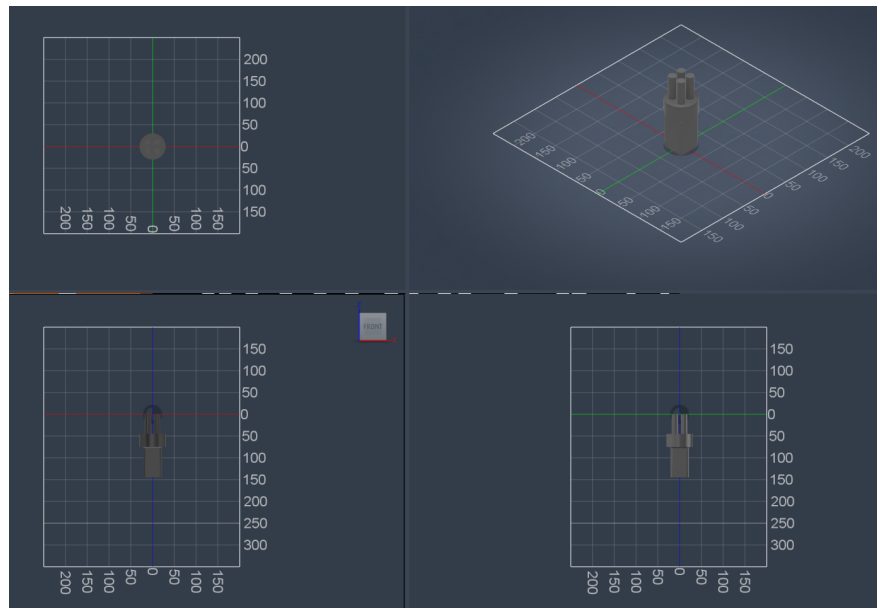


Figura 19: Diseño del eslabón 6 (herramienta) en *Fusion 360*.

7.3. Diseño alternativo: apertura unidireccional de eslabones

En robots de morfología similar al [UR3e](#), los eslabones se disponen de manera tal que, visto desde el plano lateral, el brazo adopta una forma en Z : el eslabón 2 se extiende en un sentido y el eslabón 3 se pliega en sentido opuesto, de forma que los giros de ambas articulaciones se compensan parcialmente entre sí. Esto implica que el aprovechamiento del rango articular de cada junta no es aditivo en la dirección de extensión del brazo.

Como alternativa, se diseñó una segunda versión del robot en la que todos los eslabones se abren en el mismo sentido de giro, adoptando una morfología en C vista desde el plano lateral. De este modo, el aporte de cada articulación al alcance del efector final es aditivo, lo que permite un mejor aprovechamiento del rango articular disponible.

Cabe destacar que esta modificación es estrictamente geométrica: la parametrización cinemática del robot no varía, dado que la definición del modelo Denavit-Hartenberg depende de la disposición de los ejes articulares y no de la forma particular de los eslabones. En consecuencia, toda la cinemática directa e inversa desarrollada en la sección 5 es igualmente válida para ambos diseños.

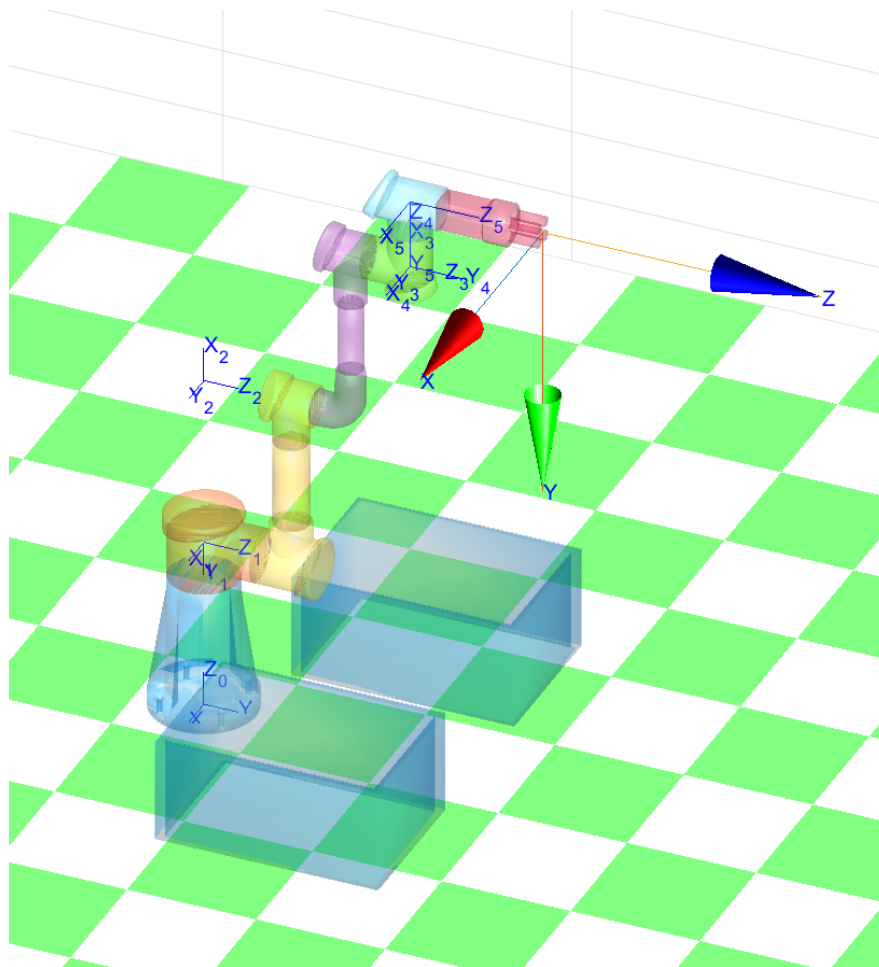


Figura 20: Modelo del robot con diseño alternativo de apertura unidireccional renderizado en *MATLAB*.

8. Simulación y verificación de tarea

9. Integración en entorno ROS2

A. Verificación y Validación de Módulos de Software de MATLAB

A.1. Cinemática Inversa

La verificación del correcto funcionamiento del módulo de cinemática inversa se realiza mediante el script `test_inv_kinematics.m`. Este incluye 3 funcionalidades

1. **Verificación redundante:** Se define una coordenada articular, se calcula la cinemática directa para obtener la coordenada cartesiana del efector final, sobre esta última se calcula la cinemática inversa con el módulo desarrollado. Finalmente, se presentan los resultados dejando en evidencia la coordenada articular inicial y el conjunto obtenido mediante cinemática inversa, también se calcula la cinemática directa para cada solución obtenida permitiendo otro contraste de resultados.

```
% ==INPUT ==
% Coordenadas articulares ingresadas:
q = [1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078];
% Las coordenadas cartesianas consigna (calculadas mediante DK) son:
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
% ==OUTPUT==
% Las coordenadas articulares solución obtenidas de la cinemática
    inversa son (solo algunas):
1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078
-4.4474    0.4922   -1.9151   -0.5557    1.7815    1.2078
-4.4474   -0.2860   -1.5354    2.9844    4.5017    4.3494
3.2637    0.2942   -4.8523   -3.1284    1.9973    2.7157
% Las coordenadas cartesianas solución obtenidas son las siguientes (se
    presentan solo algunas, todas coinciden):
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
x = -0.16918 | y = -0.19848 | z = 0.29694 | alpha = 2.7989 | beta =
    -0.3074 | gamma = 2.2087
```

2. **Cálculo directo:** Esta funcionalidad no se define específicamente para realizar una verificación sino para conocer las coordenadas articulares solución de la cinemática inversa a partir de la coordenada cartesiana ingresada de forma directa. Los resultados se presentan de misma forma al ítem 1 dado que solo se realiza un *override* durante ejecución de la coordenada cartesiana de entrada.
3. **Verificación cíclica:** Esta funcionalidad permite, hasta que se detenga la ejecución del programa, verificar cíclicamente puntos aleatorios mediante un esquema de verificación similar al del ítem 1. De forma adicional, se calcula la norma entre coordenadas cartesianas de salida y de llegada, ante una diferencia mayor a un umbral determinado en cualquiera de las soluciones pertenecientes al conjunto de solución obtenido, se detiene el programa para verificar el error y poder analizarlo.

```
% ==INPUT ==
% LOOP -> Probando posición articular
-3.2460   -1.2075   -5.0711   -4.6248    5.5550   -6.2832

% LOOP -> Las coordenadas cartesianas consigna son:
x = 0.10246 | y = -0.40972 | z = 0.45372 | alpha = 3.0035 | beta =
    0.83783 | gamma = 1.3635
% ==OUTPUT==
% Las coordenadas articulares solución obtenidas de la cinemática
    inversa son (solo algunas):
3.0372    0.7669   -1.7414   -0.5041    0.7282   -3.1416
```

```
-3.2460    -5.5163    4.5418    5.7791    -5.5550    -3.1416
-3.2460    -6.2786    5.0711    -3.4127    -0.7282     0
-2.2848     0.0199    0.9305    -3.2374     1.0729    -1.9493
% Resultado de norma:
OK
OK
OK
OK
```

NOTA: En los resultados presentados, se mencionan solo una pequeña porción de toda la información devuelta por el script.

A.2. Singularidades

Para verificar que los patrones que conducen a singularidades determinados visualmente son correctos, se elabora el script `test_singularities.m`. En él se encuentra un ciclo *while* infinito, solo terminable mediante finalización de ejecución.

Para la singularidad de codo y de muñeca, se definen valores aleatorios para todas las variables articulares que no estén involucradas en el patrón de la singularidad, para las que sí están involucradas, se define un valor aleatorio dentro del conjunto de aquellos valores que puedan provocar la singularidad según las condiciones definidas en 5.4.

Para la singularidad de hombro, debido a la presencia de 3 incógnitas en una misma ecuación, simplemente se busca verificar la observación, para esto, se dan valores aleatorios dentro de un rango de ± 10 y con signo opuesto para q_2 y q_3 , finalmente se busca el valor de q_4 que cumpla con la condición de proyección nula. El motivo por el que los valores para q_2 y q_3 están dentro del rango ± 10 y de signo opuesto, es para evitar que la proyección no sea anulable geoméricamente mediante q_4 .

Para cada caso, se calcula el jacobiano en la configuración determinada y su determinante, solo en caso de que el determinante del jacobiano sea mayor al valor de umbral, de modo que no se considere cercano a una singularidad, se detiene la ejecución y se muestra información, caso contrario, la ejecución se repite cíclicamente informando cuando se completa cada ciclo. Un ciclo equivale al análisis de una configuración para el codo, una para la muñeca y una para el hombro.

A continuación se detalla un ejemplo para cada salida.

```
% == OUTPUT DE SINGULARIDAD VERIFICADA (TODAS CONFIGURACIONES SINGULARES)
==
Ciclo completo - todas las posiciones fueron singulares - continuando...
% == OUTPUT DE NO SINGULARIDAD ==
Posición articular aleatoria - no singular - hombro
1.7748    -0.1075     0.0635     0.4317    -0.3929     3.0684
Determinante del Jacobiano: 7.897e-20
-> Detención por posición no singular - enter para continuar
```

El último ejemplo, correspondiente a una salida negativa, se produjo exigiendo un determinante de jacobiano excesivamente pequeño para que la verificación falle intencionalmente.

A.3. Generación de trayectorias

En la generación de trayectorias se buscan verificar dos aspectos fundamentales: que la trayectoria generada efectivamente interpole entre los puntos vía definidos y que el perfil de velocidades y aceleraciones resultante respete los límites cinemáticos establecidos. Para esto se elabora el script `test_gen_traj.m` que contiene el siguiente procedimiento de verificación: Se definen manualmente una serie de puntos vía con sus respectivos modos de interpolación (articular o cartesiano) y se genera la trayectoria utilizando el módulo desarrollado y se verifica:

1. **Interpolación entre puntos vía:** Se verifica que la trayectoria resultante efectivamente pase por cada uno de los puntos vía definidos, para esto, se calcula la distancia entre cada punto vía y el punto de la trayectoria correspondiente al mismo tiempo, verificando que esta distancia sea menor a un umbral determinado. En caso de detectar una cercanía lo suficientemente pequeña, se considera que el punto vía ha sido interpolado correctamente y se incrementa un contador. La

verificación será positiva siempre que el contador sea mayor o igual al número total de puntos vía definidos.

2. **Cumplimiento de límites cinemáticos:** Se imprime la comparativa entre valores máximos, perfiles cinemáticos iniciales y perfiles cinemáticos escalados para cumplir con límites. Se verifica: velocidad y aceleración angular de articulaciones, velocidad y aceleración lineal y angular de efector final. De este modo, la verificación será visual.

A.4. Nombre del Módulo 1

A.5. Nombre del Módulo 1

B. Códigos MATLAB

A continuación se presentan los códigos fuente de todos los scripts y módulos de desarrollo propio. Se excluyen los archivos de terceros ubicados en el directorio `third_party/`.

Estructura del proyecto

```
matlab/  
  main.m  
  config.m  
  flags.m  
  models/  
    robot_1.m  
  modules/  
    gen_traj.m  
    ik_comparison.m  
    inv_kinematics.m  
    scale.m  
    sym_1.m  
    tarea.m  
  tests/  
    test_gen_traj.m  
    test_inv_kinematics.m  
    test_singularities.m  
    test_workspace.m  
  visualization/  
    grafQ.m  
    grafQaE.m  
    grafScaling.m  
    myAnimate.m  
  third_party/ (no incluido)
```

Interrelación entre módulos



DIAGRAMA_MODULOS.png

Figura 21: Interrelación entre los módulos del proyecto MATLAB.

B.1. main.m

Punto de entrada del programa. Agrega al path todos los directorios del proyecto, inicializa el modelo del robot y, en función de las flags definidas en `flags.m`, ejecuta los módulos habilitados.

```
clc; clear; close all;

addpath( ./third_party/common );
addpath( ./third_party/rtb );
addpath( ./third_party/smtb );
addpath( ./models );
addpath( ./modules );
addpath( ./tests );
addpath( ./visualization );

%% =====

flags;
robot_1;

if CAD_DESIGN == 2
    cad_path = fullfile(pwd, 'models', 'CAD2');
else
    cad_path = fullfile(pwd, 'models', 'CAD');
end
```



```
%% =====

disp(== INICIANDO PROGRAMA ==);

if TEST_WORKSPACE

    test_workspace(R);

end

if TEST_INV_KINEMATICS

    test_inv_kinematics(R, TEST_LOOP_IK, PRINT_SOLUTIONS);

end

if SINGULARITIES

    test_singularities(R);

end

if TEST_GEN_TRAJ

    test_gen_traj(R, cad_path);

end

if TAREA

    tarea(R, cad_path);

end

if IK_COMPARISON

    cad_path_nb = fullfile(pwd, 'models', 'CAD_NO_BOXES');
    ik_comparison(R, cad_path_nb);

end
```

B.2. flags.m

Panel de control del flujo de ejecución. Define variables booleanas que determinan qué módulos se activan al correr `main.m`, sin necesidad de modificar el programa principal.

```
%% Flags de funcionalidades
TEST_INV_KINEMATICS = 0; % Prueba ejemplo de funcionamiento de la
    cinemática inversa
TEST_LOOP_IK = 0; % Requiere habilitar la anterior, testea en loop
    posiciones aleatorias.
PRINT_SOLUTIONS = 0; % Imprime soluciones obtenidas, geométricas y
    periódicas.
TEST_WORKSPACE = 0; % Prueba del espacio de trabajo
SINGULARITIES = 0; % Busca y verifica puntos singulares
TEST_GEN_TRAJ = 1; % Prueba ejemplo de la generación de
    trayectorias
TAREA = 0; % Tarea principal
IK_COMPARISON = 0; % Visualización de periodicidad articular de
    soluciones IK

%% Flags de opciones
CAD_DESIGN = 1; % Diseño 3D: 1 = models/CAD | 2 = models/CAD2
```

B.3. config.m

Centraliza los parámetros del sistema: velocidades y aceleraciones máximas articulares y cartesianas, paso de tiempo de la trayectoria y otras constantes. Es consultado por los módulos que requieren estos valores.

```
function cfg = config()

% C lculo de trayectorias

cfg.traj_fps=120;
cfg.traj_dt=1/cfg.traj_fps;
cfg.s_dot_max = 1.875; % Propiedad de perfil quintic
cfg.s_ddot_max = 7.5; % Propiedad de perfil quintic

% Visualizaci n y animaciones

cfg.anim_fps = 30;
cfg.anim_dt = 1/cfg.anim_fps;

% Otros par metros del robot

cfg.qdmax = ones(1,6)*pi;
cfg.qddmax = ones(1,6)*13.9626;
cfg.vmax = 1;
cfg.amax = 1.5;
cfg.wmax = 3;
cfg.alphamax = 3;

end
```

B.4. robot_1.m

Define el modelo cinemático del robot a partir de los parámetros Denavit-Hartenberg y genera el objeto R utilizado por todos los módulos del proyecto.

```
d_b = 0.250; % ajustar
d_1 = d_b;
d_2 = 0;
d_3 = 0.150+0.105+0.040;
d_4 = 0;
d_5 = 0.100;
d_6 = 0;
TCP = 0.145+0.045;% Longitud gripper + distancia origen sistema 5 a
punto de colocaci n de la tool

a_1 = 0;
a_2 = 0.250;
a_3 = 0.250;
a_4 = 0;
a_5 = 0;
a_6 = 0;

% DH 1
dh = [
    0      d_1      a_1      -pi/2;
    0      d_2      a_2      0;
    0      d_3      a_3      0;
    0      d_4      a_4      pi/2;
    0      d_5      a_5      -pi/2;
    0      d_6      a_6      0
```

```

];

q_offset = [0, -pi/2, 0, pi/2, 0, 0];
q_dmax = [pi pi pi pi pi pi]; % Velocidades máximas de cada
    articulaci n
q_ddmax = [2*pi 2*pi 2*pi 2*pi 2*pi 2*pi];

R = SerialLink(dh, 'name', 'DR - 1');
R.offset = q_offset;
base = transl(0,0,0);
tool = transl(0,0,TCP);
R.base = base;
R.tool = tool;
R.q_dmax = q_dmax;
R.q_ddmax = q_ddmax;

R.qlim(1,1:2) = [-360, 360]*pi/180;
R.qlim(2,1:2) = [-360, 360]*pi/180;
R.qlim(3,1:2) = [-360, 360]*pi/180;
R.qlim(4,1:2) = [-360, 360]*pi/180;
R.qlim(5,1:2) = [-360, 360]*pi/180;
R.qlim(6,1:2) = [-1e6,1e6]*pi/180;

q_0 = [15,20,-15,20,45,5]*pi/180;
q_0 = [-pi/5 -pi/5 -pi/5 -pi/5 -pi/5 -pi/5];
q_0 = [0 0 0 0 0 0];

% path = [pwd, '/models/CAD2'];

% R.plot3d(q_0, ...
%     'path', path, 'alpha', 0.25);
% axis equal
% axis manual
% hold on
% trplot(R.A(1:5, q_0).T, 'frame', '5', 'length', 0.1)
% hold on
% trplot(R.A(1:4, q_0).T, 'frame', '4', 'length', 0.08)
% hold on
% trplot(R.A(1:3, q_0).T, 'frame', '3', 'length', 0.05)
% hold on
% trplot(R.A(1:2, q_0).T, 'frame', '2', 'length', 0.05)
% hold on
% trplot(R.A(1:1, q_0).T, 'frame', '1', 'length', 0.05)
% hold on
% trplot(R.A(1:0, q_0).T, 'frame', '0', 'length', 0.05)
% camproj orthographic
% campos([2 2 1.5]) % posici n de la c m ara
% camtarget([0 0 0.5]) % hacia d nde mira
% camup([0 0 1]) % eje vertical

% camva(5) % zoom

% camlight headlight
% lighting gouraud

```

B.5. inv_kinematics.m

Implementa la solución analítica de cinemática inversa desarrollada en la sección 5.3. Dado un punto cartesiano objetivo, devuelve el conjunto completo de configuraciones articulares que lo alcanzan,

incluyendo soluciones periódicas.

```
function [ampl_all_sol, q_mejor] = inv_kinematics(x_t,y_t,z_t,alpha
    ,beta,gamma, q_previo, verbose, R, pose_config)

if nargin < 10 % Valor por defecto de configuraci n
    pose_config = 'NONE';
end

% Singularidades
% Mu eca: Ocurre cuando 4 y 5 son paralelos -> 5 = 0 +-180
    +-360
% Codo: Se forma un plano por las articulaciones 2, 3 y 4 -> 3 =
    0
% Hombro: El punto de intersecci n entre ejes 5 y 6 se coloca
    sobre el plano que pasa por las articulaciones 1 y 2.

eps = 10^-6;

base = [R.base.n R.base.o R.base.a R.base.t; [0 0 0 1]];
tool = [R.tool.n R.tool.o R.tool.a R.tool.t; [0 0 0 1]];

T_06 = transl(x_t,y_t,z_t) * trotx(gamma)*troty(beta)*trotx(alpha);
    % RPY ZYX
T_06_full = T_06;
T_06 = base \ T_06_full / tool;
r_11 = T_06(1,1);
r_12 = T_06(1,2);
r_13 = T_06(1,3);
r_21 = T_06(2,1);
r_22 = T_06(2,2);
r_23 = T_06(2,3);
r_31 = T_06(3,1);
r_32 = T_06(3,2);
r_33 = T_06(3,3);

x = T_06(1,4);
y = T_06(2,4);
z = T_06(3,4);

d_1 = R.links(1).d;
d_3 = R.links(3).d;
d_5 = R.links(5).d;

a_2 = R.links(2).a;
a_3 = R.links(3).a;

qlim_1_inf = R.qlim(1,1);
qlim_1_sup = R.qlim(1,2);
qlim_2_inf = R.qlim(2,1);
qlim_2_sup = R.qlim(2,2);
qlim_3_inf = R.qlim(3,1);
qlim_3_sup = R.qlim(3,2);
qlim_4_inf = R.qlim(4,1);
qlim_4_sup = R.qlim(4,2);
qlim_5_inf = R.qlim(5,1);
qlim_5_sup = R.qlim(5,2);
qlim_6_inf = R.qlim(6,1);
qlim_6_sup = R.qlim(6,2);

%% Inicializo variables
```

```
all_sol = [];  
  
q_11 = 0;  
q_12 = 0;  
q_611 = 0;  
q_612 = 0;  
q_621 = 0;  
q_622 = 0;  
q_511 = 0;  
q_512 = 0;  
q_521 = 0;  
q_522 = 0;  
q_21 = 0;  
q_22 = 0;  
q_23 = 0;  
q_24 = 0;  
q_25 = 0;  
q_26 = 0;  
q_27 = 0;  
q_28 = 0;  
q_31 = 0;  
q_32 = 0;  
q_33 = 0;  
q_34 = 0;  
q_35 = 0;  
q_36 = 0;  
q_37 = 0;  
q_38 = 0;  
q_41 = 0;  
q_42 = 0;  
q_43 = 0;  
q_44 = 0;  
q_45 = 0;  
q_46 = 0;  
q_47 = 0;  
q_48 = 0;  
A = 0;  
B = 0;  
a = 0;  
b = 0;  
C = 0;  
D = 0;  
E = 0;  
num = 0;  
den = 0;  
  
s1_valid = 1;  
s2_valid = 1;  
s3_valid = 1;  
s4_valid = 1;  
s5_valid = 1;  
s6_valid = 1;  
s7_valid = 1;  
s8_valid = 1;  
s9_valid = 1;  
s10_valid = 1;  
s11_valid = 1;  
s12_valid = 1;  
s13_valid = 1;  
s14_valid = 1;  
s15_valid = 1;
```

```
s16_valid = 1;

q_mejor = q_previo;

%% 1 Determinar q1

A = y;
B = -x;
C = -d_3;

t_11 = (-2*B+sqrt(4*B^2-4*(C-A)*(C+A)))/(2*(C-A));
t_12 = (-2*B-sqrt(4*B^2-4*(C-A)*(C+A)))/(2*(C-A));

q_11 = 2*atan(t_11);
q_12 = 2*atan(t_12);

if ~isreal(q_11)
    s1_valid = 0;
    s2_valid = 0;
    s3_valid = 0;
    s4_valid = 0;
else
    q_11 = mod(q_11-qlim_1_inf, qlim_1_sup-qlim_1_inf) + qlim_1_inf
        ;
end

if ~isreal(q_12)
    s5_valid = 0;
    s6_valid = 0;
    s7_valid = 0;
    s8_valid = 0;
else
    q_12 = mod(q_12-qlim_1_inf, qlim_1_sup-qlim_1_inf) + qlim_1_inf
        ;
end

%% 2 Determinar q6

A = -(r_22*cos(q_11)-r_12*sin(q_11));
B = r_21*cos(q_11)-r_11*sin(q_11);

if isreal(A) && isreal(B)
    q_611 = atan2(A,B);
    q_611 = mod(q_611-qlim_6_inf, qlim_6_sup-qlim_6_inf) +
        qlim_6_inf;
    q_612 = mod(q_611-qlim_6_inf+pi, qlim_6_sup-qlim_6_inf) +
        qlim_6_inf;
else
    s1_valid = 0;
    s2_valid = 0;
    s3_valid = 0;
    s4_valid = 0;
end

A = -(r_22*cos(q_12)-r_12*sin(q_12));
B = r_21*cos(q_12)-r_11*sin(q_12);

if isreal(A) && isreal(B)
```

```

q_621 = atan2(A,B);
q_621 = mod(q_621-qlim_6_inf, qlim_6_sup-qlim_6_inf) +
    qlim_6_inf;
q_622 = mod(q_621-qlim_6_inf+pi, qlim_6_sup-qlim_6_inf) +
    qlim_6_inf;

else
    s5_valid = 0;
    s6_valid = 0;
    s7_valid = 0;
    s8_valid = 0;
end

if ~isreal(q_611)
    s1_valid = 0;
    s2_valid = 0;
end
if ~isreal(q_612)
    s3_valid = 0;
    s4_valid = 0;
end
if ~isreal(q_621)
    s5_valid = 0;
    s6_valid = 0;
end
if ~isreal(q_622)
    s7_valid = 0;
    s8_valid = 0;
end

%% 3 Determinar q5

num = cos(q_611)*(r_21*cos(q_11)-r_11*sin(q_11))-sin(q_611)*(r_22*
    cos(q_11)-r_12*sin(q_11));
den = r_23*cos(q_11)-r_13*sin(q_11);

if isreal(num) && isreal(den) && s1_valid && s2_valid
    q_511 = atan2(num,den);
    q_511 = mod(q_511-qlim_5_inf, qlim_5_sup-qlim_5_inf) +
        qlim_5_inf;
else
    s1_valid = 0;
    s2_valid = 0;
end

num = cos(q_612)*(r_21*cos(q_11)-r_11*sin(q_11))-sin(q_612)*(r_22*
    cos(q_11)-r_12*sin(q_11));
den = r_23*cos(q_11)-r_13*sin(q_11);

if isreal(num) && isreal(den) && s3_valid && s4_valid
    q_512 = atan2(num,den);
    q_512 = mod(q_512-qlim_5_inf, qlim_5_sup-qlim_5_inf) +
        qlim_5_inf;
else
    s3_valid = 0;
    s4_valid = 0;
end

num = cos(q_621)*(r_21*cos(q_12)-r_11*sin(q_12))-sin(q_621)*(r_22*
    cos(q_12)-r_12*sin(q_12));
den = r_23*cos(q_12)-r_13*sin(q_12);

```

```

if isreal(num) && isreal(den) && s5_valid && s6_valid
    q_521 = atan2(num,den);
    q_521 = mod(q_521-qlim_5_inf, qlim_5_sup-qlim_5_inf) +
        qlim_5_inf;
else
    s5_valid = 0;
    s6_valid = 0;
end

num = cos(q_622)*(r_21*cos(q_12)-r_11*sin(q_12))-sin(q_622)*(r_22*
    cos(q_12)-r_12*sin(q_12));
den = r_23*cos(q_12)-r_13*sin(q_12);

if isreal(num) && isreal(den) && s7_valid && s8_valid
    q_522 = atan2(num,den);
    q_522 = mod(q_522-qlim_5_inf, qlim_5_sup-qlim_5_inf) +
        qlim_5_inf;
else
    s7_valid = 0;
    s8_valid = 0;
end

if ~isreal(q_511)
    s1_valid = 0;
    s2_valid = 0;
end
if ~isreal(q_512)
    s3_valid = 0;
    s4_valid = 0;
end
if ~isreal(q_521)
    s5_valid = 0;
    s6_valid = 0;
end
if ~isreal(q_522)
    s7_valid = 0;
    s8_valid = 0;
end

%% 4 Determinar q2

% sol 1 - 2

if s1_valid && s2_valid
    A = (r_13*cos(q_11)+r_23*sin(q_11))/(-sin(q_511));
    try
        B = -r_33/-sin(q_511);
    catch
        s1_valid = 0;
        s2_valid = 0;
    end

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_11)-y*sin(q_11);

    C = a_2^2+a^2+b^2-a_3^2;
    D = 2*a_2*b;
    E = -2*a_2*a;

    t_21 = (-2*D+sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));
    t_22 = (-2*D-sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));

```



```

        q_21 = 2*atan(t_21);
        q_22 = 2*atan(t_22);
    end

    % sol 3 - 4

    if s3_valid && s4_valid

        A = (r_13*cos(q_11)+r_23*sin(q_11))/(-sin(q_512));
        try
            B = -r_33/-sin(q_512);
        catch
            s3_valid = 0;
            s4_valid = 0;
        end

        a = -d_5*A+z-d_1;
        b = d_5*B-x*cos(q_11)-y*sin(q_11);

        C = a_2^2+a^2+b^2-a_3^2;
        D = 2*a_2*b;
        E = -2*a_2*a;

        t_23 = (-2*D+sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));
        t_24 = (-2*D-sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));

        q_23 = 2*atan(t_23);
        q_24 = 2*atan(t_24);
    end

    % sol 5 - 6

    if s5_valid && s6_valid

        A = (r_13*cos(q_12)+r_23*sin(q_12))/(-sin(q_521));
        try
            B = -r_33/-sin(q_521);
        catch
            s5_valid = 0;
            s6_valid = 0;
        end

        a = -d_5*A+z-d_1;
        b = d_5*B-x*cos(q_12)-y*sin(q_12);

        C = a_2^2+a^2+b^2-a_3^2;
        D = 2*a_2*b;
        E = -2*a_2*a;

        t_25 = (-2*D+sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));
        t_26 = (-2*D-sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));

        q_25 = 2*atan(t_25);
        q_26 = 2*atan(t_26);
    end

    % sol 7 - 8

    if s7_valid && s8_valid

        A = (r_13*cos(q_12)+r_23*sin(q_12))/(-sin(q_522));

```

```
try
    B = -r_33/-sin(q_522);
catch
    s7_valid = 0;
    s8_valid = 0;
end

a = -d_5*A+z-d_1;
b = d_5*B-x*cos(q_12)-y*sin(q_12);

C = a_2^2+a^2+b^2-a_3^2;
D = 2*a_2*b;
E = -2*a_2*a;

t_27 = (-2*D+sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));
t_28 = (-2*D-sqrt(4*D^2-4*(C-E)*(C+E)))/(2*(C-E));

q_27 = 2*atan(t_27);
q_28 = 2*atan(t_28);

end

if ~isreal(q_21)
    s1_valid = 0;
else
    q_21 = mod(q_21-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
    ;
end
if ~isreal(q_22)
    s2_valid = 0;
else
    q_22 = mod(q_22-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
    ;
end
if ~isreal(q_23)
    s3_valid = 0;
else
    q_23 = mod(q_23-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
    ;
end
if ~isreal(q_24)
    s4_valid = 0;
else
    q_24 = mod(q_24-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
    ;
end
if ~isreal(q_25)
    s5_valid = 0;
else
    q_25 = mod(q_25-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
    ;
end
if ~isreal(q_26)
    s6_valid = 0;
else
    q_26 = mod(q_26-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
    ;
end
if ~isreal(q_27)
    s7_valid = 0;
else
```

```

        q_27 = mod(q_27-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
        ;
    end
    if ~isreal(q_28)
        s8_valid = 0;
    else
        q_28 = mod(q_28-qlim_2_inf, qlim_2_sup-qlim_2_inf) + qlim_2_inf
        ;
    end

%% 5 Determinar q3

% sol 1 - 2

if s1_valid

    A = (r_13*cos(q_11)+r_23*sin(q_11))/(-sin(q_511));
    B = -r_33/-sin(q_511);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_11)-y*sin(q_11);

    num = -a_2*sin(q_21)-b;
    den = -a_2*cos(q_21)+a;

    q23 = atan2(num,den);

    q_31 = q23 - q_21;

end

if s2_valid

    A = (r_13*cos(q_11)+r_23*sin(q_11))/(-sin(q_511));
    B = -r_33/-sin(q_511);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_11)-y*sin(q_11);

    num = -a_2*sin(q_22)-b;
    den = -a_2*cos(q_22)+a;

    q23 = atan2(num,den);

    q_32 = q23 - q_22;

end

% sol 3 - 4

if s3_valid

    A = (r_13*cos(q_11)+r_23*sin(q_11))/(-sin(q_512));
    B = -r_33/-sin(q_512);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_11)-y*sin(q_11);

    num = -a_2*sin(q_23)-b;
    den = -a_2*cos(q_23)+a;

```

```
q23 = atan2(num,den);

q_33 = q23 - q_23;

end

if s4_valid

    A = (r_13*cos(q_11)+r_23*sin(q_11))/(-sin(q_512));
    B = -r_33/-sin(q_512);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_11)-y*sin(q_11);

    num = -a_2*sin(q_24)-b;
    den = -a_2*cos(q_24)+a;

    q23 = atan2(num,den);

    q_34 = q23 - q_24;

end

% sol 5 - 6

if s5_valid

    A = (r_13*cos(q_12)+r_23*sin(q_12))/(-sin(q_521));
    B = -r_33/-sin(q_521);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_12)-y*sin(q_12);

    num = -a_2*sin(q_25)-b;
    den = -a_2*cos(q_25)+a;

    q23 = atan2(num,den);

    q_35 = q23 - q_25;

end

if s6_valid

    A = (r_13*cos(q_12)+r_23*sin(q_12))/(-sin(q_521));
    B = -r_33/-sin(q_521);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_12)-y*sin(q_12);

    num = -a_2*sin(q_26)-b;
    den = -a_2*cos(q_26)+a;

    q23 = atan2(num,den);

    q_36 = q23 - q_26;

end

% sol 7 - 8
```

```

if s7_valid

    A = (r_13*cos(q_12)+r_23*sin(q_12))/(-sin(q_522));
    B = -r_33/-sin(q_522);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_12)-y*sin(q_12);

    num = -a_2*sin(q_27)-b;
    den = -a_2*cos(q_27)+a;

    q23 = atan2(num,den);

    q_37 = q23 - q_27;

end

if s8_valid

    A = (r_13*cos(q_12)+r_23*sin(q_12))/(-sin(q_522));
    B = -r_33/-sin(q_522);

    a = -d_5*A+z-d_1;
    b = d_5*B-x*cos(q_12)-y*sin(q_12);

    num = -a_2*sin(q_28)-b;
    den = -a_2*cos(q_28)+a;

    q23 = atan2(num,den);

    q_38 = q23 - q_28;

end

q_31 = mod(q_31-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_32 = mod(q_32-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_33 = mod(q_33-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_34 = mod(q_34-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_35 = mod(q_35-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_36 = mod(q_36-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_37 = mod(q_37-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;
q_38 = mod(q_38-qlim_3_inf, qlim_3_sup-qlim_3_inf) + qlim_3_inf;

%% 6 Determinar q4

% Sol 1

num = r_33;
den = -(r_13*cos(q_11)+r_23*sin(q_11));

if s1_valid
    q234 = atan2(num,den);
    q_41 = q234 - q_21 - q_31;
end

% Sol 2

num = r_33;
den = -(r_13*cos(q_11)+r_23*sin(q_11));

if s2_valid

```

```
        q234 = atan2(num,den);
        q_42 = q234 - q_22 - q_32;
    end

    % Sol 3

    num = r_33;
    den = -(r_13*cos(q_11)+r_23*sin(q_11));

    if s3_valid
        q234 = atan2(num,den);
        q_43 = (q234 - q_23 - q_33)+pi;
    end

    % Sol 4

    num = r_33;
    den = -(r_13*cos(q_11)+r_23*sin(q_11));

    if s4_valid
        q234 = atan2(num,den);
        q_44 = q234 - q_24 - q_34+pi;
    end

    % Sol 5

    num = r_33;
    den = -(r_13*cos(q_12)+r_23*sin(q_12));

    if s5_valid
        q234 = atan2(num,den);
        q_45 = q234 - q_25 - q_35;
    end

    % Sol 6

    num = r_33;
    den = -(r_13*cos(q_12)+r_23*sin(q_12));

    if s6_valid
        q234 = atan2(num,den);
        q_46 = q234 - q_26 - q_36;
    end

    % Sol 7

    num = r_33;
    den = -(r_13*cos(q_12)+r_23*sin(q_12));

    if s7_valid
        q234 = atan2(num,den);
        q_47 = q234 - q_27 - q_37+pi;
    end

    % Sol 8

    num = r_33;
    den = -(r_13*cos(q_12)+r_23*sin(q_12));

    if s8_valid
```

```
        q234 = atan2(num,den);
        q_48 = q234 - q_28 - q_38+pi;
    end

    q_41 = mod(q_41-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_42 = mod(q_42-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_43 = mod(q_43-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_44 = mod(q_44-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_45 = mod(q_45-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_46 = mod(q_46-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_47 = mod(q_47-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;
    q_48 = mod(q_48-qlim_4_inf, qlim_4_sup-qlim_4_inf) + qlim_4_inf;

    if s1_valid
        all_sol = [q_11 q_21 q_31 q_41 q_511 q_611];
        if verbose
            disp( S1 valida )
        end
    end

    if s2_valid
        all_sol = [all_sol; q_11 q_22 q_32 q_42 q_511 q_611];
        if verbose
            disp( S2 valida )
        end
    end

    if s3_valid
        all_sol = [all_sol; q_11 q_23 q_33 q_43 q_512 q_612];
        if verbose
            disp( S3 valida )
        end
    end

    if s4_valid
        all_sol = [all_sol; q_11 q_24 q_34 q_44 q_512 q_612];
        if verbose
            disp( S4 valida )
        end
    end

    if s5_valid
        all_sol = [all_sol; q_12 q_25 q_35 q_45 q_521 q_621];
        if verbose
            disp( S5 valida )
        end
    end

    if s6_valid
        all_sol = [all_sol; q_12 q_26 q_36 q_46 q_521 q_621];
        if verbose
            disp( S6 valida )
        end
    end

    if s7_valid
        all_sol = [all_sol; q_12 q_27 q_37 q_47 q_522 q_622];
        if verbose
            disp( S7 valida )
        end
    end
```

```
if s8_valid
    all_sol = [all_sol; q_12 q_28 q_38 q_48 q_522 q_622];
    if verbose
        disp( S8 valida )
    end
end

if verbose && isempty(all_sol)

    disp( No se encontraron soluciones para la IK. )
end

%% Aplicaci n de soluciones por periodicidad

if ~isempty(all_sol)

    ampl_all_sol = zeros(length(all_sol(:,1))*64,6);
    idx_max = length(all_sol(:,1));
    comb = dec2bin(0:63) - '0';

    for i=1:idx_max

        q_aux = [0 0 0 0 0 0];

        for j=1:6

            if all_sol(i,j) > 0
                q_aux(j) = all_sol(i,j) -2*pi;
            else
                q_aux(j) = all_sol(i,j) +2*pi;
            end

        end

        for k=1:64

            for l=1:6

                if comb(k,l) == 0

                    ampl_all_sol(k+64*(i-1), l) = all_sol(i,l);
                else

                    ampl_all_sol(k+64*(i-1), l) = q_aux(l);

                end

            end

        end

    end

end

%% C lculo de mejor soluci n

if ~isempty(all_sol)
```



```
        q_mejor = costo_simple(q_previo, ampl_all_sol, all_sol,
                               pose_config);

    else

        q_mejor = q_previo;
        ampl_all_sol = q_previo;
    end

%% ALTERNATIVA NUM RICA

% if isempty(all_sol)
%     if verbose
%         disp( No se encontraron soluciones mediante m todo
%             algebr ico, probando m todo num rico.)
%     end
%
%     try
%         q_numerica = R.ikine(T_06_full, q_previo, 'tol', 1e-2, '
%             ilimit', 4000, 'pinv_daming',1e-3);
%         all_sol = q_numerica;
%     catch
%         disp( No se encontraron soluciones num ricas )
%     end
%     if isempty(all_sol)
%         if verbose
%             disp( No se encontraron soluciones num ricas )
%         end
%     end
% end
% end

end

function q_mejor = costo_simple(q_previo, ampl_all_sol, all_sol,
                                pose_config)

q_mejor = q_previo;
costo_mejor = inf;

if pose_config == FORCE

    for i = 1:size(all_sol,1)

        dq = wrapToPi(all_sol(i,:) - q_previo);

        costo_nuevo = sum(abs(dq));

        if costo_nuevo < costo_mejor
            q_mejor = q_previo + dq;
            costo_mejor = costo_nuevo;
        end

    end

end
```

```

else

    for i = 1:size(ampl_all_sol,1)

        dq = wrapToPi(ampl_all_sol(i,:) - q_previo);

        costo_nuevo = sum(abs(dq));

        if costo_nuevo < costo_mejor
            q_mejor = q_previo + dq;
            costo_mejor = costo_nuevo;
        end

    end

end

end

```

B.6. gen_traj.m

Genera la trayectoria completa del robot interpolando entre los puntos vía definidos. Para cada tramo determina el modo de interpolación (articular o cartesiano) y calcula el escalado de velocidades necesario para respetar los límites cinemáticos.

```

function [q_traj, dbg] = gen_traj(R, cpoints_input, q0, qdmax,
    qddmax, verbose, vmax, amax, wmax, alphamax)

%% Inicializaci n

cfg = config();
dt = cfg.traj_dt;

q_traj = [];
dbg.q_before = [];
dbg.q_after = [];

%% Desglose de argumentos

cpoints = cpoints_input(:,1:6);
modes = cpoints_input(:,7);

%% Interpolaciones

t_acc = max(qdmax ./ qddmax);
idx_end = -1;
first = FORCE ;

for i = 1:length(cpoints(:,1))

    %% Modo articular (0)
    if modes(i) == 0 && i > idx_end

        idx_end = i;
        if i < length(cpoints(:,1))
            for j = i+1:length(cpoints(:,1))
                if modes(j) == 1; break; end
                if modes(j) == 0; idx_end = idx_end + 1; end
            end

```

```

end

% Construcción de waypoints articulares
WP      = zeros(idx_end - i + 2, 6);
WP(1,:) = get_q_ref(q_traj, q0);
q_previo = WP(1,:);

for j = i:idx_end
    [~, q_mejor] = inv_kinematics(cpoints(j,1), cpoints(j,2), cpoints(j,3), ...
                                cpoints(j,4), cpoints(j,5), cpoints(j,6),
                                ...
                                q_previo, verbose, R, first);
    first      = NONE;
    q_previo   = q_mejor;
    WP(j - i + 2,:) = q_mejor;
end

% Trayectoria inicial
q_before = mstraj(WP(2:end,:), qdmax, [], WP(1,:), dt, t_acc');

% Escala para cumplir l mites en ambos espacios
scale = compute_scale(R, q_before, qdmax, qddmax, vmax, amax, wmax, alphamax, dt);

% qdmax/k alarga la duraci n de segmentos; t_acc*k alarga las rampas
% -> velocidad escala 1/k, aceleraci n escala 1/k (correcto)
q_after = mstraj(WP(2:end,:), qdmax / scale, [], WP(1,:), dt, (t_acc * scale)');

q_traj      = [q_traj; q_after];
dbg.q_before = [dbg.q_before; q_before];
dbg.q_after  = [dbg.q_after; q_after];

%% Modo cartesiano (1)
elseif modes(i) == 1

    if i == 1
        T_q0 = R.fkine(q0);
        T_0  = [T_q0.n, T_q0.o, T_q0.a, T_q0.t; 0 0 0 1];
    else
        T_0 = transl(cpoints(i-1,1), cpoints(i-1,2), cpoints(i-1,3)) * ...
              rpy2tr(cpoints(i-1,4), cpoints(i-1,5), cpoints(i-1,6));
    end

    T_1 = transl(cpoints(i,1), cpoints(i,2), cpoints(i,3)) * ...
          rpy2tr(cpoints(i,4), cpoints(i,5), cpoints(i,6));

    tf = calculate_tf(T_0, T_1, vmax, amax, wmax, alphamax);

    if tf > 0

        q_ref = get_q_ref(q_traj, q0);

        % Trayectoria cartesiana inicial -> articular

```

```

T_temp    = make_ctrj(T_0, T_1, tf, dt);
q_before  = ctrj_to_jtrj(R, T_temp, q_ref, verbose,
    first);
first     = NONE ;

% Escala y regeneraci n si hay violaciones
scale = compute_scale(R, q_before, qdmax, qddmax, vmax,
    amax, wmax, alphamax, dt);

if scale > 1
    tf      = tf * scale;
    T_temp  = make_ctrj(T_0, T_1, tf, dt);
    q_after = ctrj_to_jtrj(R, T_temp, q_ref, verbose
        , NONE );
else
    q_after = q_before;
end

q_traj     = [q_traj;      q_after];
dbg.q_before = [dbg.q_before; q_before];
dbg.q_after  = [dbg.q_after;  q_after];

end

end

end

end

%% -- Helpers
-----

function q_ref = get_q_ref(q_traj, q0)
    if isempty(q_traj); q_ref = q0; else; q_ref = q_traj(end,:);
    end
end

function T_temp = make_ctrj(T_0, T_1, tf, dt)
    t      = 0:dt:tf;
    tau    = t / tf;
    s      = 10*tau.^3 - 15*tau.^4 + 6*tau.^5;
    T_temp = ctrj(T_0, T_1, s);
end

function qtraj = ctrj_to_jtrj(R, T_temp, q_ref, verbose, first)
    N      = size(T_temp, 3);
    qtraj  = zeros(N, 6);
    q_previo = q_ref;
    for k = 1:N
        rpy = tr2rpy(T_temp(:, :, k), 'zyx');
        [~, q_mejor] = inv_kinematics(T_temp(1,4,k), T_temp(2,4,k),
            T_temp(3,4,k), ...
                rpy(1), rpy(2), rpy(3),
                q_previo, verbose, R,
                first);

        first      = NONE ;
        q_previo   = q_mejor;
        qtraj(k,:) = q_mejor;
    end
end
end

```

```

%% -- C mputo de escala requerida
-----

function scale = compute_scale(R, qtraj, qdmax, qddmax, vmax, amax,
    wmax, alphamax, dt)
% Retorna k >= 1 tal que la trayectoria re-escalada cumple todos
% los limites.
% Al estirar el tiempo por k: velocidades escalan 1/k,
% aceleraciones 1/k^2.

N = size(qtraj, 1);

qdtraj = zeros(N, 6);
qddtraj = zeros(N, 6);
for j = 1:6
    qdtraj(:,j) = gradient(qtraj(:,j), dt);
    qddtraj(:,j) = gradient(qdtraj(:,j), dt);
end

% Espacio articular
k_qd = max(max(abs(qdtraj), [], 1) ./ qdmax);
k_qdd = sqrt(max(max(abs(qddtraj), [], 1) ./ qddmax));

% Espacio cartesiano: velocidad y aceleraci n reales via Jacobiano
v_lin = zeros(N,1); v_ang = zeros(N,1);
a_lin = zeros(N,1); a_ang = zeros(N,1);

for i = 1:N
    J = R.jacob0(qtraj(i,:));
    cv = J * qdtraj(i,:)';
    ca = J * qddtraj(i,:)';
    v_lin(i) = norm(cv(1:3));
    v_ang(i) = norm(cv(4:6));
    a_lin(i) = norm(ca(1:3));
    a_ang(i) = norm(ca(4:6));
end

k_v = max(v_lin) / vmax;
k_w = max(v_ang) / wmax;
k_a = sqrt(max(a_lin) / amax);
k_alpha = sqrt(max(a_ang) / alphamax);

scale = max([1, k_qd, k_qdd, k_v, k_w, k_a, k_alpha]);
end

%% -- Tiempo m nimo para trayectoria cartesiana
-----

function tf = calculate_tf(T_0, T_1, vmax, amax, wmax, alphamax)

cfg = config();
d = norm(T_1(1:3,4) - T_0(1:3,4));

if d < 1e-4
    tf = -1;
    return
end

[theta, ~] = tr2angvec(T_0 \ T_1);

tf_v = (cfg.s_dot_max * d) / vmax;

```

```
tf_a      = sqrt((cfg.s_ddot_max * d)      / amax);
tf_w      =      (cfg.s_dot_max * theta) / wmax;
tf_alpha = sqrt((cfg.s_ddot_max * theta) / alphamax);

tf = max([tf_v, tf_a, tf_w, tf_alpha]);
end
```

B.7. tarea.m

Define y ejecuta la tarea de *bin-to-bin transfer*: establece los puntos vía de la trayectoria, invoca al generador de trayectorias y lanza la animación del robot ejecutando el ciclo completo de la tarea.

```
function [] = tarea(R, cad_path)

cfg = config();
dt = cfg.traj_dt;
qdmax = cfg.qdmax;
qddmax = cfg.qddmax;
vmax = cfg.vmax;
amax = cfg.amax;
wmax = cfg.wmax;
alphamax = cfg.alphamax;

d = pi/180;
e=0.01;

caja_origen_superior = [-0.22, 0.3, 0.2 0*d+e 180*d+e
0*d+e];
caja_origen_inferior = [-0.22, 0.3, 0.01 0*d+e 180*d+e
0*d+e];
aux_1 = [-0.1, 0.3, 0.22 0*d+e 180*d+e
0*d+e];
punto_medio = [0, 0.3, 0.25 0*d+e 180*d+e 0*
d+e];
aux_2 = [0.1, 0.3, 0.22 0*d+e 180*d+e 0*
d+e];
caja_destino_superior = [0.22, 0.3, 0.2 0*d+e 180*d+e
0*d+e];
caja_destino_inferior = [0.22, 0.3, 0.01 0*d+e 180*d+e
0*d+e];

q_punto_medio = [0.5579 0.1345 1.8660 2.7257 1.5741
-1.0230];

% Comienzo desde punto aleatorio
q1 = R.qlim(1,1) + (R.qlim(1,2)-R.qlim(1,1))*rand;
q2 = R.qlim(2,1) + (R.qlim(2,2)-R.qlim(2,1))*rand;
q3 = R.qlim(3,1) + (R.qlim(3,2)-R.qlim(3,1))*rand;
q4 = R.qlim(4,1) + (R.qlim(4,2)-R.qlim(4,1))*rand;
q5 = R.qlim(5,1) + (R.qlim(5,2)-R.qlim(5,1))*rand;
q6 = -2*pi + (2*pi-2*pi)*rand;

q0 = q_punto_medio;
% -1.0600 0.2519 -1.8082 -0.0185 1.5844 -2.6408
% -1.0600 -1.4227 1.8082 -1.9602 1.5844 -2.6408
% -1.0600 -0.5988 -0.8019 2.9676 -1.5844 0.5008
% -1.0600 -1.3561 0.8019 2.1210 -1.5844 0.5008
% 1.0439 1.3607 -0.8079 -2.1100 1.5673 -0.5370
% 1.0439 0.5979 0.8079 -2.9628 1.5673 -0.5370
% 1.0439 1.4190 -1.8038 1.9692 -1.5673 2.6046
% 1.0439 -0.2518 1.8038 0.0325 -1.5673 2.6046
%
```

```
% q0 = q_punto_medio;
trayectoria = [
    punto_medio 0 %
    % aux_1 0 %
    caja_origen_superior 0 %
    caja_origen_inferior 1
    caja_origen_superior 1
    % % aux_1 0 %
    punto_medio 0 %
    % % aux_2 0 %
    caja_destino_superior 0 %
    caja_destino_inferior 1
    caja_destino_superior 1
    % % aux_2 0 %
    punto_medio 0
];

[q_traj, dbg] = gen_traj(R, trayectoria, q0, qdmax, qddmax, 0, vmax
    , amax, wmax, alphamax);

%% Visualizaci n
myAnimate(R, q_traj, cad_path);

grafQaE(R, q_traj);

qd_traj = zeros(size(q_traj));
qdd_traj = zeros(size(q_traj));
for j = 1:size(q_traj,2)
    qd_traj(:,j) = gradient(q_traj(:,j), dt);
    qdd_traj(:,j) = gradient(qd_traj(:,j), dt);
end

grafQ(q_traj, qd_traj, qdd_traj);

grafScaling(R, dbg, qdmax, vmax, amax, wmax, alphamax);

end
```

B.8. ik_comparison.m

Permite comparar visualmente diferentes soluciones de cinemática inversa para una misma pose del efector final, animando cada configuración articular sobre el modelo CAD del robot.

```
function [] = ik_comparison(R, cad_path)
%
% Visualiza las variantes periodicas ( 2 en q1, q2, q3) de una
% soluci n
% geom trica de la cinem tica inversa para la pose de referencia:
%
% q_ref = [0 , 45 , 45 , 45 , 45 , 45 ]
%
% Genera 2^3 = 8 combinaciones (v lidas dentro de qlim) y anima el
% robot
% movi ndose entre ellas en espacio articular. Al llegar a cada
% posici n
% se habilita el bot n CONTINUAR en la figura.

cfg = config();
dt = cfg.traj_dt;
step = floor(cfg.traj_fps / cfg.anim_fps);
qdmax = cfg.qdmax;
t_acc = max(qdmax ./ cfg.qddmax);
```

```

d = pi / 180;
ws = [-0.6 0.6 -0.6 0.6 -0.1 0.8];

%% 1. FK de la pose de referencia
q_ref = [0, 45, 45, 45, 45, 45] * d;
T = R.fkine(q_ref);
p = transl(T);
rpy = tr2rpy(T, 'zyx');
x = p(1); y = p(2); z = p(3);
alpha = rpy(1); beta = rpy(2); gamma = rpy(3);

fprintf(' [IK_COMPARISON] Pose cartesiana: x=%.3f y=%.3f z=%.3f\n'
, x, y, z);

%% 2. IK y extracci n de soluci n geom trica base
[ampl_all_sol, ~] = inv_kinematics(x, y, z, alpha, beta, gamma,
q_ref, false, R, 'FORCE');

if isempty(ampl_all_sol)
    error(' [IK_COMPARISON] La IK no encontr soluciones para la
pose de referencia. ');
end

% ampl_all_sol: 64 filas por soluci n geom trica.
% La fila 64*(i-1)+1 corresponde a la soluci n geom trica i sin
shifts.
geo_sols = ampl_all_sol(1:64:end, :);
costs = sum(abs(wrapToPi(geo_sols - q_ref)), 2);
[~, best] = min(costs);
q_geo = geo_sols(best, :);

fprintf(' [IK_COMPARISON] Soluci n geom trica base (la m s
cerca a q_ref):\n');
fprintf(' [ '); fprintf('%.1f ', q_geo * 180/pi); fprintf(']\n');

%% 3. Variantes peri dicas en q1, q2, q3 (mismo criterio que
inv_kinematics)
% Si q_geo(j) >= 0: alternativa = q_geo(j) - 2
% Si q_geo(j) < 0: alternativa = q_geo(j) + 2
q_alt = q_geo;
for j = 1:3
    if q_geo(j) >= 0
        q_alt(j) = q_geo(j) - 2*pi;
    else
        q_alt(j) = q_geo(j) + 2*pi;
    end
end

solutions = [];
sol_labels = {};

for combo = 0:7
    bits = dec2bin(combo, 3) - '0'; % [b1 b2 b3], MSB = q1
    q_cand = q_geo;
    for j = 1:3
        if bits(j)
            q_cand(j) = q_alt(j);
        end
    end
end

```



```

        if all(q_cand >= R.qlim(:,1)' & q_cand <= R.qlim(:,2)')
            solutions(end+1, :) = q_cand;           %#ok<AGROW>
            sol_labels{end+1} = make_label(q_cand, q_geo, bits);  %#
                ok<AGROW>
        end
    end

n_sol = size(solutions, 1);
fprintf('[IK_COMPARISON] Variantes validas: %d / 8\n', n_sol);

if n_sol == 0
    error('[IK_COMPARISON] Ninguna variante periodica cae dentro
        de los limites articulares.');
```

end

%% 4. Figura

```

n_joints = R.n;
qlim      = R.qlim;
warn_frac = 0.10;
bar_h     = 0.65;

fig = figure('Name', 'IK Comparison Periodicidad Articular', ...
    'Color', [0.96 0.96 0.96], 'NumberTitle', 'off');

% Eje invisible para el titulo dinámico (arriba del robot)
ax_title = axes(fig, 'Position', [0.16 0.93 0.82 0.06], 'Visible',
    'off');
ax_joints = axes(fig, 'Position', [0.01 0.12 0.13 0.80]);
ax_robot = axes(fig, 'Position', [0.16 0.12 0.82 0.80]);

h_title = text(ax_title, 0.5, 0.5, 'Iniciando...', ...
    'HorizontalAlignment', 'center', 'VerticalAlignment', 'middle',
    ...
    'FontSize', 11, 'FontWeight', 'bold', 'Color', 'k', 'Units', '
        normalized');
```

```

btn = uicontrol(fig, 'Style', 'pushbutton', ...
    'Units', 'normalized', 'Position', [0.35 0.02 0.30 0.08], ...
    'String', 'CONTINUAR', 'FontSize', 12, 'FontWeight', 'bold',
    ...,
    'BackgroundColor', [0.20 0.72 0.20], 'ForegroundColor', 'w', ...
    'Enable', 'off', ...
    'Callback', @(~,~) set(fig, 'UserData', 'go'));
```

%% 5. Panel de articulaciones (izquierda)

```

hold(ax_joints, 'on');
set(ax_joints, ...
    'YDir', 'reverse', ...
    'XLim', [0 1], 'YLim', [0.5 n_joints + 0.5], ...
    'XTick', [0 0.5 1], 'XTickLabel', {'min', '', 'max'}, ...
    'YTick', 1:n_joints, 'YTickLabel', {'q_1', 'q_2', 'q_3', 'q_4',
        'q_5', 'q_6'}, ...
    'Color', [0.95 0.95 0.95], 'Box', 'on', 'FontSize', 8);
title(ax_joints, {'ARTICULACIONES', '\color{rgb}{0.2,0.45,0.85}
    } con variantes periodicas'}, ...
    'FontSize', 7.5, 'FontWeight', 'bold', 'Color', 'k');
```

```

hJMarkers = gobjects(n_joints, 1);
for j = 1:n_joints
    ylo = j - bar_h/2;
    yhi = j + bar_h/2;
    q_n = joint_norm(solutions(1,j), qlim(j,:));
```

```

% Zonas de advertencia en los extremos del rango
patch(ax_joints, [0 warn_frac warn_frac 0], [ylo
    ylo yhi yhi], [1.0 0.80 0.80], 'EdgeColor','none');
patch(ax_joints, [1-warn_frac 1 1 1-warn_frac], [ylo
    ylo yhi yhi], [1.0 0.80 0.80], 'EdgeColor','none');
% Track de fondo
patch(ax_joints, [0 1 1 0], [ylo yhi yhi], [0.82 0.82 0.82], 'EdgeColor',[0.55 0.55
    0.55], 'LineWidth',0.5);
% Linea central de referencia (posicion media del rango)
plot(ax_joints, [0.5 0.5], [ylo yhi], '--', 'Color',[0.65 0.65
    0.65], 'LineWidth',0.8);
% Borde azul para q1, q2, q3: indica que tienen variante
    periodica
if j <= 3
    plot(ax_joints, [0 1 1 0 0], [ylo ylo yhi yhi ylo], '-', ...
        'Color',[0.20 0.45 0.85], 'LineWidth', 2.0);
end
% Marcador de posicion actual (verde rojo segun
    proximidad al limite)
t0 = limit_t(q_n, warn_frac);
hJMarkers(j) = plot(ax_joints, [q_n q_n], [ylo yhi], '-', ...
    'Color', bar_color(t0), 'LineWidth', 4);
end

%% 6. Setup del robot en la primera posicion
axes(ax_robot);
R.plot3d(solutions(1,:), 'workspace', ws, 'path', cad_path, 'delay'
    , 0);
axes(ax_robot);
axis(ax_robot, 'equal');
xlim(ax_robot, ws(1:2)); ylim(ax_robot, ws(3:4)); zlim(ax_robot,
    ws(5:6));
axis(ax_robot, 'vis3d');
set(ax_robot, 'CameraViewAngleMode', 'manual');
camlight headlight;
lighting gouraud;

%% 7. Loop principal: animar y esperar al usuario en cada solucion
for k = 1:n_sol

    btn_travel(btn);
    set(h_title, 'String', sprintf('Viajando a solucion %d / %d...',
        k, n_sol));
    drawnow;

    if k == 1
        % Primera posicion: robot ya esta ahi desde el setup
        R.animate(solutions(1,:));
        update_joints(hJMarkers, solutions(1,:), qlim, warn_frac);
    else
        % Trayectoria en espacio articular con mstraj (no pasa por
            IK)
        q_seg = mstraj(solutions(k,:), qdmax, [], solutions(k-1,:),
            dt, t_acc');
        for i = 1:step:size(q_seg, 1)
            if ~isvalid(fig); return; end
            R.animate(q_seg(i,:));
            update_joints(hJMarkers, q_seg(i,:), qlim, warn_frac);
            drawnow;
            pause(cfg.anim_dt);
        end
    end
end

```

```

        end
    end

    % Asegurar posici n exacta final y actualizar t tulo
    R.animate(solutions(k,:));
    update_joints(hJMarkers, solutions(k,:), qlim, warn_frac);
    set(h_title, 'String', sprintf('Soluci n %d / %d %s', k,
        n_sol, sol_labels{k}));
    drawnow;

    % Habilitar bot n o finalizar
    if k < n_sol
        btn_ready(btn);
        set(fig, 'UserData', ''); % resetear antes de
            esperar
        waitfor(fig, 'UserData', 'go');
        if ~isvalid(fig); return; end
    else
        btn_end(btn);
    end

end

end

%%
=====

% Funciones locales
%%
=====

function btn_ready(btn)
set(btn, 'Enable','on', 'String','CONTINUAR ', ...
    'BackgroundColor',[0.20 0.72 0.20], 'ForegroundColor','w');
end

function btn_travel(btn)
set(btn, 'Enable','off', 'String','Viajando...', ...
    'BackgroundColor',[0.74 0.74 0.74], 'ForegroundColor',[0.40
        0.40 0.40]);
end

function btn_end(btn)
set(btn, 'Enable','off', 'String','FIN', ...
    'BackgroundColor',[0.30 0.45 0.85], 'ForegroundColor','w');
end

function update_joints(hMarkers, q, qlim, warn_frac)
for j = 1:length(q)
    q_n = joint_norm(q(j), qlim(j,:));
    set(hMarkers(j), 'XData', [q_n q_n], 'Color', bar_color(limit_t
        (q_n, warn_frac)));
end
end

% Label para el t tulo: indica si cada joint usa el valor base o
    la variante 360
function label = make_label(q_cand, q_geo, bits)
names = {'q1', 'q2', 'q3'};
parts = cell(1, 3);

```

```
for j = 1:3
    val_deg = round(q_cand(j) * 180/pi);
    base_deg = round(q_geo(j) * 180/pi);
    if bits(j) == 0
        parts{j} = sprintf('%s: %d (base)', names{j}, val_deg);
    elseif q_cand(j) > q_geo(j)
        parts{j} = sprintf('%s: %d (%d +360 )', names{j},
            val_deg, base_deg);
    else
        parts{j} = sprintf('%s: %d (%d -360 )', names{j},
            val_deg, base_deg);
    end
end
label = strjoin(parts, ' ');
end

function n = joint_norm(q, lim)
n = max(0, min(1, (q - lim(1)) / (lim(2) - lim(1))));
end

function t = limit_t(q_norm, warn_frac)
t = max(0, min(1, 1 - min(q_norm, 1 - q_norm) / warn_frac));
end

function c = bar_color(t)
s = sqrt(t);
c = [s, 1-s, 0];
end
```

B.9. scale.m

Escala los archivos STL exportados desde *Fusion 360* para que sus dimensiones coincidan con las definidas en el modelo DH, garantizando la correcta visualización del robot en el entorno de simulación.

```
% --- CONFIGURACION ---
base_dir = fullfile(fileparts(mfilename('fullpath')), '..', '
models');
scale_factor = 1000; % metros mm (usar 1 si ya est n en mm)

designs = {'CAD', 'CAD2', 'CAD_NO_BOXES'};

stl_files = {
    'link0.stl', ...
    'link1.stl', ...
    'link2.stl', ...
    'link3.stl', ...
    'link4.stl', ...
    'link5.stl', ...
    'link6.stl'
};

% --- PROCESAMIENTO ---
total = 0;

for d = 1:length(designs)

    input_folder = fullfile(base_dir, designs{d}, 'originales');
    output_folder = fullfile(base_dir, designs{d});

    if ~exist(input_folder, 'dir')
        fprintf('Carpeta no encontrada, saltando: %s\n',
            input_folder);
    end
end
```

```

        continue;
    end

    if ~exist(output_folder, 'dir')
        mkdir(output_folder);
    end

    fprintf('\n=== Procesando dise o: %s ===\n', designs{d});

    for i = 1:length(stl_files)
        filename = stl_files{i};
        input_path = fullfile(input_folder, filename);
        output_path = fullfile(output_folder, filename);

        if ~exist(input_path, 'file')
            fprintf(' Archivo no encontrado, saltando: %s\n',
                filename);
            continue;
        end

        fprintf(' Procesando: %s\n', filename);

        TR = stlread(input_path);
        vertices_scaled = TR.Points / scale_factor;
        stlwrite(triangulation(TR.ConnectivityList, vertices_scaled
            ), output_path);

        fprintf(' Guardado en: %s\n', output_path);
        total = total + 1;
    end

end

fprintf('\nListo! %d archivos escalados.\n', total);

```

B.10. sym_1.m

Realiza el desarrollo simbólico de las matrices de transformación homogénea para obtener, de forma simplificada, las expresiones algebraicas utilizadas en la resolución analítica de la cinemática inversa.

```

clc; clear; close all;
syms a b c d e f a1 a2 a3 a4 a5 a6 d1 d2 d3 d4 d5 d6 r11 r12 r13
    r21 r22 r23 r31 r32 r33 x y z

%% RHS

T12 = [ sin(b)  cos(b)  0  a2*sin(b)
        -cos(b) sin(b)  0  -a2*cos(b)
        0       0      1  0
        0       0      0  1];

T23 = [ cos(c)  -sin(c)  0  a3*cos(c);
        sin(c)  cos(c)  0  a3*sin(c);
        0       0      1  d3;
        0       0      0  1 ];

T34 = [ -sin(d)  0  cos(d)  0
        cos(d)  0  sin(d)  0
        0       1  0      0
        0       0  0      1];

T45 = [ cos(e)  0  -sin(e)  0

```

```

sin(e)  0  cos(e)  0
0        -1  0      d5
0         0  0      1];

RHS = simplify(T12 * T23 * T34 * T45);

%% LHS

T06 = [r11 r12 r13 x
       r21 r22 r23 y
       r31 r32 r33 z
       0   0   0   1];

T01_inv = [cos(a)      sin(a)      0   0
            0           0          -1  d1
            -sin(a)     cos(a)      0   0
            0           0           0  1];

T56_inv = [ cos(f)  sin(f)  0  0
            -sin(f) cos(f)  0  0
            0        0      1  0
            0        0      0  1];

LHS = simplify(T01_inv*T06*T56_inv);

```

B.11. test_workspace.m

Verifica el espacio de trabajo del robot generando configuraciones de máximo estiramiento y trazando la región alcanzable por el efector final. Los resultados se presentan en la sección 5.2.

```

function [] = test_workspace(R)

step = 5*pi/180;

q1 = R.qlim(1,1):step:R.qlim(1,2);
q2 = pi/2;
q3 = 0;
q4 = -pi/2;
q5 = -pi/2;
q6 = 0;

puntos_alcanzados = zeros(numel(q1),2);
idx = 0;

for i = 1:numel(q1)

    idx = idx + 1;

    q = [q1(i) q2 q3 q4 q5 q6];

    T = R.fkine(q);
    p = T.t;

    x = p(1);
    y = p(2);

    puntos_alcanzados(idx,:) = [x y];

end

```

```
figure;

% Plot puntos
plot(puntos_alcanzados(:,1), puntos_alcanzados(:,2), ...
     Marker , * , LineStyle , none );
hold on;

% Centro del workspace
xc = mean(puntos_alcanzados(:,1));
yc = mean(puntos_alcanzados(:,2));

% Par metro angular
theta = linspace(0, 2*pi, 200);

% Radios
r_max = 0.5;           % 500 mm
r_tarea = 0.34515;    % 345.15 mm

% C rculo rango m ximo
x_max = xc + r_max*cos(theta);
y_max = yc + r_max*sin(theta);
plot(x_max, y_max, 'r-', 'LineWidth', 1.5);

% C rculo alcance tarea
x_t = xc + r_tarea*cos(theta);
y_t = yc + r_tarea*sin(theta);
plot(x_t, y_t, 'g--', 'LineWidth', 1.5);

% Labels
title( Comprobaci n gruesa del espacio de trabajo SB-S4 );
xlabel( x [m] );
ylabel( y [m] );

% Mantener proporci n
axis equal;

grid minor;

legend( Puntos alcanzados , ...
        Rango m ximo (500 mm) , ...
        Alcance tarea (345.15 mm) , ...
        Location , best );

hold off;

end
```

B.12. test_inv_kinematics.m

Verifica el correcto funcionamiento del módulo de cinemática inversa mediante las tres modalidades descritas en el Apéndice A.1: verificación redundante, cálculo directo y verificación cíclica con puntos aleatorios.

```
function [] = test_inv_kinematics(R ,TEST_LOOP_IK , PRINT_SOLUTIONS)
disp(' ');
disp(---> TEST_INV_KINEMATICS )

d = pi/180;
e = 0.01;
q_tik = [1.8358    0.4922   -1.9151   -0.5557    1.7815    1.2078];
%R.plot(q_tik)
```

```
q_previo = [29 29 29 29 29 29]*d;
disp( Posici n articular previa )
disp(q_previo)

disp( Probando posici n articular: );
disp(q_tik)
T_tik = R.fkine(q_tik);

p_tik= transl(T_tik);
rpy_tik = tr2rpy(T_tik,'zyx');

x_tik = p_tik(1);
y_tik = p_tik(2);
z_tik = p_tik(3);

alpha_tik = rpy_tik(1);
beta_tik = rpy_tik(2);
gamma_tik = rpy_tik(3);

% override
% x_tik = 0;
% y_tik = 0.3;
% z_tik = 0.25;
% %
% alpha_tik = 0*d+e;
% beta_tik = 180*d+e;
% gamma_tik = 0*d+e;

coordenadas_cartesianas = x = +string(x_tik)+ | y = +string(
    y_tik)+ | z = +string(z_tik)+ | alpha = +string(alpha_tik)+
    | beta = +string(beta_tik)+ | gamma = +string(gamma_tik);

disp( Las coordenadas cartesianas consigna son: )
disp(coordenadas_cartesianas)

[sol_tik, q_mejor] = inv_kinematics(x_tik,y_tik,z_tik,alpha_tik,
    beta_tik,gamma_tik, q_previo, 1, R);

if ~isempty(sol_tik)

    disp( Las coordenadas articulares soluci n obtenidas de la
        cinematica inversa son: )
    disp(sol_tik);
    disp( Las coordenadas cartesianas soluci n obtenidas son las
        siguientes )

    for i=1:length(sol_tik(:,1))

        T_tik_sol = R.fkine(sol_tik(i,:));

        p_tik_sol = transl(T_tik_sol);
        rpy_tik_sol = tr2rpy(T_tik_sol,'zyx');

        x_tik_sol = p_tik_sol(1);
        y_tik_sol = p_tik_sol(2);
        z_tik_sol = p_tik_sol(3);

        alpha_tik_sol = rpy_tik_sol(1);
        beta_tik_sol = rpy_tik_sol(2);
        gamma_tik_sol = rpy_tik_sol(3);
```



```

        coordenadas_cartesianas = x = +string(x_tik_sol)+ | y =
            +string(y_tik_sol)+ | z = +string(z_tik_sol)+ |
            alpha = +string(alpha_tik_sol)+ | beta = +string(
                beta_tik_sol)+ | gamma = +string(gamma_tik_sol);

        disp(coordenadas_cartesianas)

    end
else

    disp( No se han encontrado soluciones v lidas );
end

disp( La mejor soluci n es )
disp(q_mejor)

%% IMPRESI N DE SOLUCIONES

% solutions(R, sol_tik);

%% LOOP TESTING

if TEST_LOOP_IK

    disp( )
    disp( )
    disp( )
    disp( LOOP -> Comenzando prueba en loop )

    while 1

        q1 = R.qlim(1,1) + (R.qlim(1,2)-R.qlim(1,1))*rand;
        q2 = R.qlim(2,1) + (R.qlim(2,2)-R.qlim(2,1))*rand;
        q3 = R.qlim(3,1) + (R.qlim(3,2)-R.qlim(3,1))*rand;
        q4 = R.qlim(4,1) + (R.qlim(4,2)-R.qlim(4,1))*rand;
        q5 = R.qlim(5,1) + (R.qlim(5,2)-R.qlim(5,1))*rand;
        q6 = -2*pi + (2*pi-2*pi)*rand;

        q_loop = [q1 q2 q3 q4 q5 q6];
        disp( LOOP -> Probando posici n articular );
        disp(q_loop);

        T_loop = R.fkine(q_loop);
        T_loop = [T_loop.n T_loop.o T_loop.a T_loop.t ; 0 0 0 1];

        p_loop= transl(T_loop);
        rpy_loop = tr2rpy(T_loop, 'zyx');

        x_loop = p_loop(1);
        y_loop = p_loop(2);
        z_loop = p_loop(3);

        alpha_loop = rpy_loop(1);
        beta_loop = rpy_loop(2);
        gamma_loop = rpy_loop(3);

        coordenadas_cartesianas = x = +string(x_loop)+ | y = +
            string(y_loop)+ | z = +string(z_loop)+ | alpha = +
            string(alpha_loop)+ | beta = +string(beta_loop)+ |
            gamma = +string(gamma_loop);

        disp( LOOP -> Las coordenadas cartesianas consigna son: )
    end
end

```

```

disp(coordenadas_cartesianas)

[sol_loop, ~] = inv_kinematics(x_loop,y_loop,z_loop,
    alpha_loop,beta_loop,gamma_loop, q_previo, 1, R)

if ~isempty(sol_loop)
    for i=1:length(sol_loop(:,1))

        T_sol = R.fkine(sol_loop(i,:));
        T_sol = [T_sol.n T_sol.o T_sol .a T_sol.t ; 0 0 0
            1];

        dp = norm(transl(T_loop) - transl(T_sol));
        dang = acos((trace(T_loop(1:3,1:3) '* T_sol(1:3,1:3))
            - 1)/2);    % diferencia angular

        if dp >= 10^-2 || dang >= 10^-2
            disp( L00P -> Posible error de IK - detenido
                para verificar )
            p_sol = transl(T_sol);
            rpy_sol = tr2rpy(T_sol,'zyx');

            x_sol = p_sol(1);
            y_sol = p_sol(2);
            z_sol = p_sol(3);

            alpha_sol = rpy_sol(1);
            beta_sol = rpy_sol(2);
            gamma_sol = rpy_sol(3);
            coordenadas_cartesianas = x = +string(x_sol)
                + | y = +string(y_sol)+ | z = +string(
                    z_sol)+ | alpha = +string(alpha_sol)+ |
                    beta = +string(beta_sol)+ | gamma = +
                    string(gamma_sol);

            disp( L00P -> Las coordenadas cartesianas
                resultantes son: )
            disp(coordenadas_cartesianas)
            input( )
        else
            disp( OK )
        end

    end

end

pause(0.5);

end

end

end

```

B.13. test_singularities.m

Verifica que las configuraciones singulares identificadas analíticamente produzcan efectivamente pérdida de rango en el Jacobiano, para los tres tipos de singularidad: hombro, codo y muñeca.

```
function [] = singularities(R)
```

```

while 1

    %% Singularidad de mu eca q5 = 0

    q1_rand = R.qlim(1,1) + (R.qlim(1,2)-R.qlim(1,1))*rand;
    q2_rand = R.qlim(2,1) + (R.qlim(2,2)-R.qlim(2,1))*rand;
    q3_rand = R.qlim(3,1) + (R.qlim(3,2)-R.qlim(3,1))*rand;
    q4_rand = R.qlim(4,1) + (R.qlim(4,2)-R.qlim(4,1))*rand;
    q5_rand = randi([-2, 2]);
    q6_rand = R.qlim(6,1) + (R.qlim(6,2)-R.qlim(6,1))*rand;

    q_wrist = [q1_rand q2_rand q3_rand q4_rand q5_rand*pi q6_rand];

    J = R.jacob0(q_wrist);
    det_J = det(J);

    if abs(det_J) > 10^-3
        disp( Posici n articular aleatoria - no singular - mu eca
            );
        disp(q_wrist);
        disp( Determinante del Jacobiano: +string(det_J));
        disp(-> Detenci n por posici n no singular - enter para
            continuar )
        input ( );
    end

    %% Singularidad de codo q3 = k*pi

    q1_rand = R.qlim(1,1) + (R.qlim(1,2)-R.qlim(1,1))*rand;
    q2_rand = R.qlim(2,1) + (R.qlim(2,2)-R.qlim(2,1))*rand;
    q3_rand = randi([-2, 2]);
    q4_rand = R.qlim(4,1) + (R.qlim(4,2)-R.qlim(4,1))*rand;
    q5_rand = R.qlim(5,1) + (R.qlim(5,2)-R.qlim(5,1))*rand;
    q6_rand = R.qlim(6,1) + (R.qlim(6,2)-R.qlim(6,1))*rand;

    q_elbow = [q1_rand q2_rand q3_rand*pi q4_rand q5_rand q6_rand];

    J = R.jacob0(q_elbow);
    det_J = det(J);

    if abs(det_J) > 10^-3
        disp( Posici n articular aleatoria - no singular - codo );
        disp(q_elbow);
        disp( Determinante del Jacobiano: +string(det_J));
        disp(-> Detenci n por posici n no singular - enter para
            continuar )
        input ( );
    end

    %% Singularidad de hombro, S5 o S6 sobre plano definido por A1
    y A2

    a2 = R.links(2).a;
    a3 = R.links(3).a;
    d5 = R.links(5).d;

    q1 = -pi + (2*pi)*rand;
    q2 = deg2rad(-10 + 20*rand);
    if q2 >= 0

```

```

        q3 = deg2rad(-10 * rand);
    else
        q3 = deg2rad(10 * rand);
    end
    q4 = -pi + (2*pi)*rand;
    q5 = -pi + (2*pi)*rand;
    q6 = -pi + (2*pi)*rand;

    q4 = asin((-a2*sin(q2)-a3*sin(q2+q3))/(d5))-q2-q3;

    % 4. Armar el vector y evaluar
    q_shoulder = [q1 q2 q3 q4 q5 q6];
    J = R.jacob0(q_shoulder);
    det_J = det(J);

    if abs(det_J) > 10^-3
        disp( Posici n articular aleatoria - no singular - hombro
            );
        disp(q_shoulder);
        disp( Determinante del Jacobiano: +string(det_J));
        disp(-> Detenci n por posici n no singular - enter para
            continuar )
        input( );
    end

    disp( Ciclo completo - todas las posiciones fueron singulares -
        continuando... )

end

```

B.14. test_gen_traj.m

Verifica el generador de trayectorias ejecutando casos de prueba predefinidos y animando el resultado sobre el modelo CAD del robot.

```

function [] = test_gen_traj(R, cad_path)

cfg = config();

d = pi/180;
fps = cfg.traj_fps;
dt = cfg.traj_dt;
qddmax = cfg.qddmax;
qddmax = cfg.qddmax;
vmax = cfg.vmax;
amax = cfg.amax;
wmax = cfg.wmax;
alphamax = cfg.alphamax;

e=0.01;

cpoints_modes = [
    0.1, 0.3, 0.22 0*d+e 180*d+e 0*d+e 0
    0.22, 0.3, 0.2 0*d+e 180*d+e 0*d+e 0
];

q0 = [1.3913 0.5478 1.0215 3.1548 1.5629 -0.1896];

qddmax = [180 180 180 360 360 360]*d;
qddmax = [800 800 800 800 800 800]*d;

```

```
[q_traj, dbg] = gen_traj(R, cpoints_modes, q0, qdmax, qddmax, 0,
    vmax, amax, wmax, alphamax);

myAnimate(R, q_traj, cad_path);

grafQaE(R, q_traj);

qd_traj = zeros(size(q_traj));
qdd_traj = zeros(size(q_traj));
for j = 1:size(q_traj,2)
    qd_traj(:,j) = gradient(q_traj(:,j), dt);
    qdd_traj(:,j) = gradient(qd_traj(:,j), dt);
end

grafQ(q_traj, qd_traj, qdd_traj)

grafScaling(R, dbg, qdmax, vmax, amax, wmax, alphamax)

%% Verificación de paso por puntos de control

N = size(q_traj,1);
N_cpoints = size(cpoints_modes,1);
N_alcanzados = 0;

for i = 1:N
    T_i = R.fkine(q_traj(i,:));

    for j = 1:N_cpoints
        T_cpoint = transl(cpoints_modes(j,1), cpoints_modes(j,2),
            cpoints_modes(j,3)) * ...
            rpy2tr(cpoints_modes(j,4), cpoints_modes(j,5),
                cpoints_modes(j,6));
        R_i = [T_i.n, T_i.o, T_i.a];
        pos_ok = norm(T_i.t - T_cpoint(1:3,4)) < e;
        rot_ok = norm(R_i - T_cpoint(1:3,1:3), 'fro') < e; %
            Norma frobenius, similar a euclidiana pero tiene pasos
            previos para llevar la matriz a un vector.
        if pos_ok && rot_ok
            N_alcanzados = N_alcanzados + 1;
            break
        end
    end
end

N_alcanzados
if N_alcanzados >= N_cpoints
    disp(Todos los puntos de control fueron alcanzados.);
else
    disp(No se alcanzaron todos los puntos de control.);
end

end
```

B.15. myAnimate.m

Genera la animación 3D del robot en tres paneles sincronizados: posiciones articulares con zonas de advertencia cerca de los límites, modelo CAD del robot en movimiento, y número de condición del jacobiano para detectar proximidad a singularidades.

```
% myAnimate Anima la trayectoria articular de un robot con
% visualizaci n diagn stica
%
% SINTAXIS
% myAnimate(R, q_traj, cad_path)
%
% DESCRIPCI N
% Reproduce la trayectoria articular q_traj sobre el modelo
% SerialLink R
% mostrando simult neamente tres paneles:
%
% [ARTICULACIONES] | [ROBOT 3D] | [JACOBIANO]
%
% Panel izquierdo Indica, para cada articulaci n, la
% posici n normalizada
% en su rango de movimiento. La zona de color
% verde claro
% corresponde a la regi n central (bien
% alejada de l mites)
% y se vuelve roja cuando el ngulo se acerca
% a un extremo.
%
% Panel central Animaci n 3D del robot con modelos CAD,
% rastro de la
% trayectoria del efector final y c mara fija
% .
% Panel derecho N mero de condici n (J) del Jacobiano
% geom trico en
% escala logar tmica. El color va de verde (
% bien condicionado)
% a rojo (cerca de singularidad). Si (J) >
% 100 aparece la
% advertencia ! SINGULAR .
%
% ENTRADAS
% R - Objeto SerialLink (Robotics Toolbox) que representa
% el robot.
% Se usa R.jacob0, R.n, R.qlim, R.plot3d y R.animate.
% q_traj - Matriz (M n_joints) de ngulos articulares en
% rad,
% muestreada a cfg.traj_fps (por defecto 120 Hz).
% cad_path - Cadena con la ruta al directorio que contiene los
% archivos
% STL/CAD para plot3d.
%
% PAR METROS INTERNOS (desde config.m)
% traj_fps = 120 Hz Frecuencia a la que fue generada q_traj
% .
% anim_fps = 30 Hz Frecuencia de renderizado de la
% animaci n.
% anim_dt = 1/30 s Pausa entre cuadros (pause real-time).
%
% MATEM TICA RELEVANTE
% Diezmado temporal
% step = floor(traj_fps / anim_fps)
% Se toma 1 de cada `step` filas de q_traj sub-muestreo por
% factor 4.
% Resultado: q_sub (N n_joints), con N M/4 cuadros.
%
% Jacobiano geom trico en la base (J ~{6 n })
% J = R.jacob0(q) v_ee = J q
```

```
% Las 3 primeras filas mapean velocidades articulares a
% velocidad lineal
% del efector; las 3 ltimas a velocidad angular.
% J depende de la configuraci n q (no-lineal en general).
%
% Descomposici n en Valores Singulares (SVD)
% J = U          V^T,      = diag( _1      _2
%      _n          0)
% Los valores singulares { _i } miden cu nto amplifica el
% Jacobiano
% movimientos articulares en cada direcci n del espacio de la
% tarea.
% _min      0 indica que existe una direcci n del espacio de
% la tarea
% que ninguna combinaci n de velocidades articulares puede
% alcanzar
% de forma eficiente: el robot est en (o cerca de) una
% singularidad.
%
% N mero de condici n
% (J) = _max / _min
%      = 1      is tropo , el robot puede moverse igual de bien
% en todas
% las direcciones configuraci n ptima .
% singularidad cinem tica; la inversa J^{-1}
% es num ricamente
% inestable y los torques pueden dispararse.
% Umbral de advertencia en este c digo:      > 100.

function [] = myAnimate(R, q_traj, cad_path)

% =====
% Par metros de muestreo
% =====
% La trayectoria fue generada a traj_fps Hz. La animaci n corre a
% anim_fps Hz.
% step = 4 (para 120 Hz / 30 Hz), es decir, se descarta 3 de cada 4
% puntos.
% Esto reduce el costo de renderizado sin degradar la percepci n
% visual
% dado que el ojo humano no distingue diferencias a m s de ~24 fps
% .
cfg = config();
step = floor(cfg.traj_fps / cfg.anim_fps);
q_sub = q_traj(1:step:end, :); % (N      n_joints) sub-muestreado
N      = size(q_sub, 1);      % cantidad de cuadros a renderizar

% Workspace (espacio de trabajo visible) en metros: [xmin xmax ymin
% ymax zmin zmax]
ws = [-0.6 0.6 -0.6 0.6 -0.1 0.8];

% =====
% Pre-c mputo: Jacobiano y an lisis de singularidades
% =====
% Se calcula UNA SOLA VEZ antes de la animaci n para no bloquear
% el loop
% de rendering con lgebra lineal costosa frame a frame.
%
% Para cada configuraci n q_sub(i,:):
% 1. Se obtiene J      ^{6 6} (Jacobiano geom trico en la
% base, frame {0}).
% 2. Se factoriza J = U      V^T (SVD completa).
```

```
% 3. Se extraen _max = sv(1) y _min = sv(end).
% 4. (J) = _max / _min (n mero de condici n en la norma
2).
% 5. _min se guarda por separado como indicador directo de
singularidad.
cond_vals = zeros(N, 1);
smin_vals = zeros(N, 1);
for i = 1:N
    J = R.jacob0(q_sub(i,:)); % J ^{6 6}
    sv = svd(J); % sv ordenado
    descendente: _1 _6
    cond_vals(i) = max(sv) / min(sv); % (J) = _max / _min
    smin_vals(i) = min(sv); % _min : proximidad a la
singularidad
end

% Rango del eje logar tmico del panel derecho.
% cond_lo = 1 (m nimo te rico: matriz perfectamente is tropa).
% cond_hi se ajusta al peor caso observado en la trayectoria, con
un m nimo
% de 1000 para que la escala sea legible incluso si el robot est
bien condicionado.
cond_lo = 1;
cond_hi = max(max(cond_vals) * 1.5, 1000);

% Umbral a partir del cual se considera que el robot est en
singularidad
% pr ctica (no necesariamente , pero num ricamente
problem tico).
sing_thresh = 100;

% Par metros del panel de articulaciones
n_joints = R.n;
qlim = R.qlim; % (n_joints 2): [q_min, q_max] por fila,
en rad
warn_frac = 0.10; % El 10% m s cercano a cada l mite se
colorea en rojo
bar_h = 0.65; % Altura visual de cada barra articular (en
unidades del eje Y)

% =====
% Figura con tres paneles
% =====
% Las posiciones se dan como [left bottom width height] en
fracci n de la figura.
% ax_joints: franja izquierda (13% del ancho)
% ax_robot: centro (62% del ancho)
% ax_bar: franja derecha (13% del ancho, centrada
verticalmente)
figure;
ax_joints = axes('Position', [0.01 0.06 0.13 0.86]);
ax_robot = axes('Position', [0.16 0.06 0.62 0.86]);
ax_bar = axes('Position', [0.83 0.22 0.13 0.62]);

% =====
% Panel izquierdo: articulaciones
% =====
% Cada articulaci n j (j = 1..n_joints) se representa como una
barra horizontal.
% El eje X del panel va de 0 a 1: representa la posici n
normalizada
% q_norm = (q - q_min) / (q_max - q_min) [0, 1]
```



```
% El eje Y indexa las articulaciones (1 arriba, n_joints abajo, con
YDir=reverse).
hold(ax_joints, 'on');
set(ax_joints, ...
    'YDir', 'reverse', ... % q_1 en la parte superior
    'XLim', [0 1], ...
    'YLim', [0.5 n_joints + 0.5], ...
    'XTick', [0 0.5 1], ...
    'XTickLabel', {'min', '', 'max'}, ... % 0=m nimo articular, 1=
    m ximo articular
    'YTick', 1:n_joints, ...
    'YTickLabel', {'q_1','q_2','q_3','q_4','q_5','q_6'}, ...
    'Color', [0.95 0.95 0.95], ...
    'Box', 'on', ...
    'FontSize', 8);
title(ax_joints, 'ARTICULACIONES', 'FontSize', 9, 'FontWeight', '
bold', 'Color', 'k');

% hJMarkers(j): handle al marcador de posici n de la articulaci n
j.
% Se actualiza en cada frame del loop de animaci n.
hJMarkers = gobjects(n_joints, 1);

for j = 1:n_joints
    % Coordenadas verticales de la barra de la articulaci n j
    yc = j;
    ylo = yc - bar_h/2;
    yhi = yc + bar_h/2;

    % Posici n inicial normalizada de la articulaci n j
    q_norm = joint_norm(q_sub(1,j), qlim(j,:));

    % Zona roja en el extremo izquierdo: q_norm [0, warn_frac]
    % Indica que la articulaci n est cerca de su l mite m nimo
    .
    patch(ax_joints, [0 warn_frac warn_frac 0], [ylo ylo yhi yhi],
    ...
    [1 0.80 0.80], 'EdgeColor', 'none');

    % Zona roja en el extremo derecho: q_norm [1-warn_frac, 1]
    % Indica que la articulaci n est cerca de su l mite m ximo
    .
    patch(ax_joints, [1-warn_frac 1 1 1-warn_frac], [ylo ylo yhi
    yhi], ...
    [1 0.80 0.80], 'EdgeColor', 'none');

    % Track de fondo: rect ngulo gris que ocupa todo el rango
    [0,1]
    patch(ax_joints, [0 1 1 0], [ylo ylo yhi yhi], ...
    [0.82 0.82 0.82], 'EdgeColor', [0.55 0.55 0.55], 'LineWidth
    ', 0.5);

    % L nea vertical en q_norm = 0.5: posici n media del rango
    articular.
    % Referencia visual para saber si la articulaci n est
    centrada.
    plot(ax_joints, [0.5 0.5], [ylo yhi], '--', ...
    'Color', [0.65 0.65 0.65], 'LineWidth', 0.8);

    % Marcador din mico: l nea vertical en la posici n actual
    q_norm.
    % Su color var a seg n limit_t (verde=centro, rojo=l mite).
```

```

t0 = limit_t(q_norm, warn_frac);
hJMarkers(j) = plot(ax_joints, [q_norm q_norm], [ylo yhi], '-',
    ...
    'Color', bar_color(t0), 'LineWidth', 4);
end

% =====
% Panel central: robot 3D
% =====
% R.plot3d renderiza el robot con modelos CAD en la configuraci n
%   q_sub(1,:).
%   'workspace'      define los l mites del volumen visible
%   'trail'          activa el rastro del TCP (extremo operacional)
%                   en rojo
%   'path'           directorio con archivos STL de los eslabones
%   'delay',0        sin retardo interno (el timing lo controla
%                   pause() abajo)
axes(ax_robot);
R.plot3d(q_sub(1,:), 'workspace', ws, 'trail', {'r','LineWidth',2},
    'path', cad_path, 'delay', 0);

% Se restaura ax_robot como eje activo porque plot3d puede crear
%   ejes internos.
axes(ax_robot);
axis(ax_robot, 'equal');
xlim(ax_robot, ws(1:2));
ylim(ax_robot, ws(3:4));
zlim(ax_robot, ws(5:6));

% 'vis3d' congela la relaci n de aspecto durante las rotaciones
%   interactivas,
%   evitando que el robot salte visualmente al girar la c mara.
axis(ax_robot, 'vis3d');

% 'manual' impide que MATLAB reajuste autom ticamente el ngulo
%   de c mara
%   entre frames, lo que causar a un efecto de zoom pulsante .
set(ax_robot, 'CameraViewAngleMode', 'manual');

% Iluminaci n: 'headlight' sigue a la c mara; 'gouraud' interpola
%   normales
%   por v rtice dando una superficie m s suave que 'flat' shading.
camlight headlight;
lighting gouraud;

% =====
% Panel derecho: Jacobiano (n mero de condici n)
% =====
% El panel muestra (J) como una barra que crece hacia arriba en
%   escala
%   logar tmica. El color va de verde ( 1) a rojo ( )
%   .
%   Escala logar tmica: permite visualizar en el mismo eje rangos
%   muy dispares.
%   Por ejemplo, puede valer 2 (excelente) o 5000 (singularidad
%   pr ctica).
%   En escala lineal, el valor 2 ser a invisible junto al 5000.
cval0 = cond_vals(1);
smin0 = smin_vals(1);
t0 = bar_t(cval0, cond_lo, cond_hi);

```

```

hold(ax_bar, 'on');

% Rect ngulo de fondo gris que ocupa todo el rango [cond_lo,
cond_hi]
patch(ax_bar, [0 1 1 0], [cond_lo cond_lo cond_hi cond_hi], ...
[0.85 0.85 0.85], 'EdgeColor', 'none');

% Barra de color din mica: su altura Y superior es cval (en escala
log)
hBar = patch(ax_bar, [0 1 1 0], [cond_lo cond_lo cval0 cval0], ...
bar_color(t0), 'EdgeColor', 'none');

% L nea horizontal que marca el nivel exacto de cval
hLine = plot(ax_bar, [0 1], [cval0 cval0], 'k-', 'LineWidth', 1.5);

set(ax_bar, ...
'YScale', 'log', ... % eje Y logar tmico: cada d cada
ocupa el mismo espacio
'XTick', [], ... % sin marcas en X (el eje X no tiene
significado aqu )
'YColor', 'k', ...
'XColor', 'k', ...
'Color', [0.95 0.95 0.95], ...
'Box', 'on', ...
'LineWidth', 1);
ylim(ax_bar, [cond_lo cond_hi]);
xlim(ax_bar, [0 1]);

title(ax_bar, 'JACOBIANO', 'Color', 'k', 'FontSize', 9, '
FontWeight', 'bold');
ylabel(ax_bar, 'cond(J)', 'Color', 'k', 'FontSize', 8);

% xlabel se reutiliza como cuadro de texto num rico (truco de
layout):
% muestra (J) redondeado y _min con 4 decimales.
hInfo = xlabel(ax_bar, sprintf('cond: %.0f\nsmin: %.4f', cval0,
smin0), ...
'Color', 'k', 'FontSize', 8, 'FontWeight', 'bold');

% Texto de advertencia de singularidad (visible s lo cuando >
sing_thresh).
% 'Units','normalized' posici n relativa al panel,
independiente de la escala Y.
hWarn = text(ax_bar, 0.5, 0.65, '! SINGULAR', ...
'Units', 'normalized', ...
'HorizontalAlignment', 'center', ...
'VerticalAlignment', 'middle', ...
'Color', [0.85 0 0], ...
'FontSize', 8, ...
'FontWeight', 'bold', ...
'Rotation', 90, ...
'Clipping', 'off', ...
'Visible', 'off');

% =====
% Loop de animaci n
% =====
% En cada iteraci n i:
% 1. R.animate mueve el modelo 3D a la configuraci n q_sub(i,:).
% 2. Se actualizan los marcadores articulares (panel izquierdo).
% 3. Se actualiza la barra de condici n del Jacobiano (panel
derecho).

```

```
% 4. drawnow fuerza el redibujado inmediato del buffer gráfico.
% 5. pause(anim_dt) sincroniza la velocidad de reproducción con
    anim_fps.
%
% Nota: se usa set() para modificar propiedades de handles
    existentes en lugar
% de borrar y recrear objetos gráficos, lo que es mucho más
    rápido (~10%).
for i = 1:N

    % Mueve el robot al frame i en el panel central
    R.animate(q_sub(i,:));

    % -- Panel izquierdo: actualizar cada marcador articular --
    for j = 1:n_joints
        % Normalización: q_norm = (q - q_min) / (q_max - q_min)
            [0,1]
        q_norm = joint_norm(q_sub(i,j), qlim(j,:));

        % Distancia al límite más cercano, mapeada a [0,1]:
        % t = 0 articulación en el centro del rango color
            verde
        % t = 1 articulación tocando un límite
            color rojo
        t = limit_t(q_norm, warn_frac);

        % XData fija la posición horizontal del marcador; Color
            cambia según t
        set(hJMarkers(j), 'XData', [q_norm q_norm], 'Color',
            bar_color(t));
    end

    % -- Panel derecho: actualizar barra de condición del
        Jacobiano --
    cval = cond_vals(i);
    smin = smin_vals(i);

    % Mapeo logarítmico de cval al intervalo [0,1]:
    % t = (log10(cval) - log10(c_min)) / (log10(c_max) - log10(
        c_min))
    % t = 0 = 1 (is tropo) verde
    % t = 1 = cond_hi rojo
    t = bar_t(cval, cond_lo, cond_hi);

    % YData: borde inferior fijo en cond_lo, borde superior en cval
        (log scale)
    set(hBar, 'YData', [cond_lo cond_lo cval cval], 'FaceColor',
        bar_color(t));
    set(hLine, 'YData', [cval cval]);
    set(hInfo, 'String', sprintf('cond: %.0f\nsmin: %.4f', cval,
        smin));

    % Advertencia de singularidad práctica
    if cval > sing_thresh
        set(hWarn, 'Visible', 'on');
    else
        set(hWarn, 'Visible', 'off');
    end

    drawnow; % flushea el pipeline gráfico de
        MATLAB
end
```

```

        pause(cfg.anim_dt);      % espera 1/30 s para mantener anim_fps =
                                30 Hz
    end

end % myAnimate

% =====
% Funciones auxiliares (locales al archivo)
% =====

% joint_norm Normaliza un ángulo articular al intervalo [0, 1]
%
%   q_norm = (q - q_min) / (q_max - q_min)
%
%   El resultado se satura a [0,1] con max/min para evitar
%   desbordamientos
%   si q excede los l mites nominales (posible en la primera/
%   ltima muestra
%   de la trayectoria por efectos num ricos de la interpolaci n).
function n = joint_norm(q, lim)
n = (q - lim(1)) / (lim(2) - lim(1));
n = max(0, min(1, n));
end

% limit_t Mapea la posici n normalizada a un nivel de alarma t
%   [0, 1]
%
%   margin = distancia normalizada al l mite m s cercano:
%   margin = min(q_norm, 1 - q_norm)      [0, 0.5]
%   margin = 0      si q_norm      {0, 1} (en el l mite)
%   margin = 0.5    si q_norm = 0.5      (en el centro)
%
%   t = 1 - margin / warn_frac
%   t = 1      si margin = 0              (en el l mite)
%   rojo
%   t = 0      si margin      warn_frac  (zona segura)
%   verde
%   t      (0,1) dentro de la zona de advertencia [0,
%   warn_frac]
%
%   La saturaci n con max(0,...) garantiza t      0 en la zona
%   central.
function t = limit_t(q_norm, warn_frac)
margin = min(q_norm, 1 - q_norm);
t = max(0, min(1, 1 - margin / warn_frac));
end

% bar_t Mapea el n mero de condici n      a t      [0, 1] en escala
%   logar tmica
%
%   Se trabaja en escala log porque      puede variar en varios
%   rdenes de magnitud.
%   La transformaci n es:
%   t = (log10( ) - log10( _min )) / (log10( _max ) - log10(
%   _min ))
%
%   Propiedades:
%   t = 0      si      = _min = 1 (robot is tropo , condici n
%   perfecta)      verde
%   t = 1      si      _max      (robot en singularidad pr ctica
%   )      rojo
%

```

```
% El max(cval, cond_lo) evita log10(0) para < 1 (no ocurre
% en la práctica
% ya que 1 siempre, pero protege ante errores numéricos
% de SVD).
% El min(..., 1) satura t en 1 si supera cond_hi.
function t = bar_t(cval, cond_lo, cond_hi)
t = min( ...
    (log10(max(cval, cond_lo)) - log10(cond_lo)) / ...
    (log10(cond_hi) - log10(cond_lo)), ...
    1);
end

% bar_color Interpola linealmente entre verde puro y rojo puro
%
% Entrada: t [0, 1]
% Salida: c = [R, G, B] en el espacio de color RGB normalizado
% [0,1]^3
%
% Mapeo:
% t = 0 c = [0, 1, 0] (verde)
% t = 1 c = [1, 0, 0] (rojo)
% t = 0.5 c = [0.5, 0.5, 0] (amarillo-oliva)
%
% Nota: el canal azul B = 0 siempre. La interpolación es lineal
% en RGB,
% no en espacio perceptual (HSV/LAB), por lo que el punto medio
% visual
% no es exactamente amarillo puro, sino un amarillo-verdoso. Para
% aplicaciones donde la percepción exacta sea crítica,
% considerar HSV.
function c = bar_color(t)
c = [t, 1-t, 0];
end
```

B.16. grafQ.m

Grafica la evolución temporal de las coordenadas articulares, velocidades y aceleraciones a lo largo de una trayectoria, permitiendo verificar visualmente el cumplimiento de los límites cinemáticos.

```
function [] = grafQ(q_traj, qd_traj, qdd_traj)

cfg = config;
dt = cfg.traj_dt;

t = 1:size(q_traj,1);
t = t*dt;

step = 10;

%% ===== POSICION =====
figure;
hold on
subplot(6,1,1)
plot(t, q_traj(:,1)); ylabel('q1 [rad]'); grid minor;
subplot(6,1,2)
plot(t, q_traj(:,2)); ylabel('q2 [rad]'); grid minor;
subplot(6,1,3)
plot(t, q_traj(:,3)); ylabel('q3 [rad]'); grid minor;
subplot(6,1,4)
plot(t, q_traj(:,4)); ylabel('q4 [rad]'); grid minor;
subplot(6,1,5)
```

```

plot(t, q_traj(:,5)); ylabel('q5 [rad]'); grid minor;
subplot(6,1,6)
plot(t, q_traj(:,6)); ylabel('q6 [rad]'); xlabel('Tiempo'); grid
minor;
sgtitle(q_{traj})
hold off

%% ===== VELOCIDAD =====
figure;
hold on
subplot(6,1,1)
plot(t, qd_traj(:,1)); ylabel('qd1 [rad/s]'); grid minor;
subplot(6,1,2)
plot(t, qd_traj(:,2)); ylabel('qd2 [rad/s]'); grid minor;
subplot(6,1,3)
plot(t, qd_traj(:,3)); ylabel('qd3 [rad/s]'); grid minor;
subplot(6,1,4)
plot(t, qd_traj(:,4)); ylabel('qd4 [rad/s]'); grid minor;
subplot(6,1,5)
plot(t, qd_traj(:,5)); ylabel('qd5 [rad/s]'); grid minor;
subplot(6,1,6)
plot(t, qd_traj(:,6)); ylabel('qd6 [rad/s]'); xlabel('Tiempo');
grid minor;
sgtitle(qd_{traj})
hold off

%% ===== ACELERACION =====
figure;
hold on
subplot(6,1,1)
plot(t, qdd_traj(:,1)); ylabel('qdd1 [rad/s^2]'); grid minor;
subplot(6,1,2)
plot(t, qdd_traj(:,2)); ylabel('qdd2 [rad/s^2]'); grid minor;
subplot(6,1,3)
plot(t, qdd_traj(:,3)); ylabel('qdd3 [rad/s^2]'); grid minor;
subplot(6,1,4)
plot(t, qdd_traj(:,4)); ylabel('qdd4 [rad/s^2]'); grid minor;
subplot(6,1,5)
plot(t, qdd_traj(:,5)); ylabel('qdd5 [rad/s^2]'); grid minor;
subplot(6,1,6)
plot(t, qdd_traj(:,6)); ylabel('qdd6 [rad/s^2]'); xlabel('Tiempo');
grid minor;
sgtitle(qdd_{traj})
hold off

end

```

B.17. grafQaE.m

Calcula y grafica la posición, velocidad y aceleración cartesianas del efector final a lo largo de la trayectoria, obtenidas mediante cinemática directa y diferenciación numérica de la trayectoria articular.

```

function [] = grafQaE(R, q_traj)

cfg = config();
dt = cfg.traj_dt;

N = size(q_traj,1);

t = (0:N-1)' * dt;

```

```
T = zeros(4,4,N);

x = zeros(N,1);
y = zeros(N,1);
z = zeros(N,1);
roll = zeros(N,1);
pitch = zeros(N,1);
yaw = zeros(N,1);

%% --- CINEMÁTICA DIRECTA ---
for i = 1:N

    T(:, :, i) = R.fkine(q_traj(i, :));

    x(i) = T(1,4,i);
    y(i) = T(2,4,i);
    z(i) = T(3,4,i);

    rpy = tr2rpy(T(:, :, i), 'zyx');

    roll(i) = rpy(1);
    pitch(i) = rpy(2);
    yaw(i) = rpy(3);

end

%% --- DERIVADAS NUMÉRICAS ---

vx = gradient(x, dt);
vy = gradient(y, dt);
vz = gradient(z, dt);

vroll = gradient(roll, dt);
vpitch = gradient(pitch, dt);
vyaw = gradient(yaw, dt);

ax = gradient(vx, dt);
ay = gradient(vy, dt);
az = gradient(vz, dt);

aroll = gradient(vroll, dt);
apitch = gradient(vpitch, dt);
ayaw = gradient(vyaw, dt);

%% --- PLOTS ---

% ===== POSICIÓN =====
figure;
subplot(6,1,1); plot(t,x); grid minor; ylabel('x [m]'); xlim([t(1) t(end)]);
subplot(6,1,2); plot(t,y); grid minor; ylabel('y [m]'); xlim([t(1) t(end)]);
subplot(6,1,3); plot(t,z); grid minor; ylabel('z [m]'); xlim([t(1) t(end)]);
subplot(6,1,4); plot(t,roll); grid minor; ylabel('roll [rad]');
    xlim([t(1) t(end)]);
subplot(6,1,5); plot(t,pitch); grid minor; ylabel('pitch [rad]');
    xlim([t(1) t(end)]);
subplot(6,1,6); plot(t,yaw); grid minor; ylabel('yaw [rad]');
    xlabel('t [s]'); xlim([t(1) t(end)]);
sgtitle('Posición cartesiana');
```



```
% ===== VELOCIDAD =====
figure;
subplot(6,1,1); plot(t,vx); grid minor; ylabel('vx [m/s]'); xlim([t
(1) t(end)]);
subplot(6,1,2); plot(t,vy); grid minor; ylabel('vy [m/s]'); xlim([t
(1) t(end)]);
subplot(6,1,3); plot(t,vz); grid minor; ylabel('vz [m/s]'); xlim([t
(1) t(end)]);
subplot(6,1,4); plot(t,vroll); grid minor; ylabel('vroll [rad/s]');
xlim([t(1) t(end)]);
subplot(6,1,5); plot(t,vpitch); grid minor; ylabel('vpitch [rad/s]'
); xlim([t(1) t(end)]);
subplot(6,1,6); plot(t,vyaw); grid minor; ylabel('vyaw [rad/s]');
xlabel('t [s]'); xlim([t(1) t(end)]);
sgtitle('Velocidad cartesiana');

% ===== ACELERACION =====
figure;
subplot(6,1,1); plot(t,ax); grid minor; ylabel('ax [m/s^2]'); xlim
([t(1) t(end)]);
subplot(6,1,2); plot(t,ay); grid minor; ylabel('ay [m/s^2]'); xlim
([t(1) t(end)]);
subplot(6,1,3); plot(t,az); grid minor; ylabel('az [m/s^2]'); xlim
([t(1) t(end)]);
subplot(6,1,4); plot(t,aroll); grid minor; ylabel('aroll [rad/s^2]'
); xlim([t(1) t(end)]);
subplot(6,1,5); plot(t,apitch); grid minor; ylabel('apitch [rad/s
^2]'); xlim([t(1) t(end)]);
subplot(6,1,6); plot(t,ayaw); grid minor; ylabel('ayaw [rad/s^2]');
xlabel('t [s]'); xlim([t(1) t(end)]);
sgtitle('Aceleracion cartesiana');

end
```

B.18. grafScaling.m

Compara las trayectorias antes y después del escalamiento, graficando velocidades articulares y magnitudes cartesianas (velocidad lineal, aceleración lineal, velocidad angular y aceleración angular) junto con sus límites admisibles, para verificar que el reescalado elimina las violaciones.

```
function grafScaling(R, dbg, qdmax, vmax, amax, wmax, alphamax)

cfg = config();
dt = cfg.traj_dt;

q_before = dbg.q_before;
q_after = dbg.q_after;

if isempty(q_before)
    disp('grafScaling: sin datos de segmentos modo 0, nada que
graficar. ');
    return;
end

N_b = size(q_before, 1);
N_a = size(q_after, 1);

t_b = (0:N_b-1)' * dt;
t_a = (0:N_a-1)' * dt;

%% --- Velocidades articulares ---
```

```

qd_b = zeros(N_b, 6);
qd_a = zeros(N_a, 6);
for j = 1:6
    qd_b(:,j) = gradient(q_before(:,j), dt);
    qd_a(:,j) = gradient(q_after(:,j), dt);
end

%% --- Velocidades del EF via Jacobiano ---

qdd_b = zeros(N_b, 6);
qdd_a = zeros(N_a, 6);
for j = 1:6
    qdd_b(:,j) = gradient(qd_b(:,j), dt);
    qdd_a(:,j) = gradient(qd_a(:,j), dt);
end

v_b      = zeros(N_b,1);  w_b      = zeros(N_b,1);
a_b      = zeros(N_b,1);  alpha_b  = zeros(N_b,1);

for i = 1:N_b
    J      = R.jacob0(q_before(i,:));
    ee_vel = J * qd_b(i,:)';
    ee_acc = J * qdd_b(i,:)';      % aproximacion: ignora Jdot*qd
    v_b(i) = norm(ee_vel(1:3));
    w_b(i) = norm(ee_vel(4:6));
    a_b(i) = norm(ee_acc(1:3));
    alpha_b(i) = norm(ee_acc(4:6));
end

v_a      = zeros(N_a,1);  w_a      = zeros(N_a,1);
a_a      = zeros(N_a,1);  alpha_a  = zeros(N_a,1);

for i = 1:N_a
    J      = R.jacob0(q_after(i,:));
    ee_vel = J * qd_a(i,:)';
    ee_acc = J * qdd_a(i,:)';
    v_a(i) = norm(ee_vel(1:3));
    w_a(i) = norm(ee_vel(4:6));
    a_a(i) = norm(ee_acc(1:3));
    alpha_a(i) = norm(ee_acc(4:6));
end

%% --- FIGURA 1: Velocidades articulares ---

labels = {'q1','q2','q3','q4','q5','q6'};

figure;
sgtitle('Velocidades articulares: efecto del escalado');
for j = 1:6
    subplot(6,1,j);
    hold on;
    plot(t_b, abs(qd_b(:,j)), 'b', 'DisplayName', 'Sin escalar');
    plot(t_a, abs(qd_a(:,j)), 'r', 'DisplayName', 'Escalado');
    yline(qdmax(j), 'k--', 'DisplayName', 'qdmax');
    ylabel([labels{j} ' [rad/s]']);
    if j == 6, xlabel('t [s]'); end
    grid minor;
    xlim([0, max(t_b(end), t_a(end))]);
    if j == 1, legend('Location','northeast'); end
    hold off;
end

```

```
%% --- FIGURA 2: Velocidades del efector final ---

figure;
sgtitle('Velocidades EF: efecto del escalado');

subplot(4,1,1);
hold on;
plot(t_b, v_b, 'b', 'DisplayName', 'Sin escalar');
plot(t_a, v_a, 'r', 'DisplayName', 'Escalado');
yline(vmax, 'k--', 'DisplayName', 'vmax');
ylabel('|v| [m/s]'); grid minor; xlim([0, max(t_b(end), t_a(end))]);
;
legend('Location','northeast'); hold off;

subplot(4,1,2);
hold on;
plot(t_b, a_b, 'b', 'DisplayName', 'Sin escalar');
plot(t_a, a_a, 'r', 'DisplayName', 'Escalado');
yline(amax, 'k--', 'DisplayName', 'amax');
ylabel('|a| [m/s^2]'); grid minor; xlim([0, max(t_b(end), t_a(end))]);
];
hold off;

subplot(4,1,3);
hold on;
plot(t_b, w_b, 'b', 'DisplayName', 'Sin escalar');
plot(t_a, w_a, 'r', 'DisplayName', 'Escalado');
yline(wmax, 'k--', 'DisplayName', 'wmax');
ylabel('|\omega| [rad/s]'); grid minor; xlim([0, max(t_b(end), t_a(end))]);
];
hold off;

subplot(4,1,4);
hold on;
plot(t_b, alpha_b, 'b', 'DisplayName', 'Sin escalar');
plot(t_a, alpha_a, 'r', 'DisplayName', 'Escalado');
yline(alphamax, 'k--', 'DisplayName', 'alphamax');
ylabel('|\alpha| [rad/s^2]'); xlabel('t [s]'); grid minor;
xlim([0, max(t_b(end), t_a(end))]);
hold off;

end
```

Referencias

- [1] Grand View Research. *Global Warehouse Robotics Market Size & Outlook, 2023–2030*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market-size/global>
- [2] Grand View Research. *Warehouse Robotics Market Outlook: United States*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/united-states>
- [3] Grand View Research. *Warehouse Robotics Market Outlook: China*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/china>
- [4] Grand View Research. *Warehouse Robotics Market Outlook: Latin America*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/latin-america>
- [5] Grand View Research. *Warehouse Robotics Market Outlook: Brazil*. 2024. Disponible en: <https://www.grandviewresearch.com/horizon/outlook/warehouse-robotics-market/brazil>
- [6] Stuart Russell & Peter Norvig. *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson, 2021.
- [7] John J. Craig. *Introduction to Robotics: Mechanics and Control* (4th ed.). Pearson, 2021.
- [8] Peter Corke. *Robotics, Vision and Control: Fundamental Algorithms in MATLAB* (2nd ed.). Springer, 2023.
- [9] Michael F. Ashby. *Materials Selection in Mechanical Design* (6th ed.). Elsevier, 2022.
- [10] Klaus-Jürgen Bathe. *Finite Element Procedures*. Prentice Hall, 2006.
- [11] Ramu Krishnan. *Permanent Magnet Brushless DC Motor Drives and Controls*. CRC Press, 2017.
- [12] Aaron Martínez & Enrique Fernández. *Learning ROS 2: Basics, Concepts, and Practical Applications*. Packt, 2024.
- [13] Jesse H. Hyman. *Learning Robotics Simulation Using ROS and Gazebo*. Packt, 2018.